

CS229: Machine Learning

Vorlesungsskript – Zusammenfassung

Andrew Ng & Tengyu Ma

Stanford University

Ausgabe 2023 – Deutsche Zusammenfassung

Inhaltsverzeichnis

I	Überwachtes Lernen	6
1	Lineare Regression	6
1.1	Aufgabenstellung	6
1.2	Kostenfunktion und Gradientenverfahren	6
1.3	Normalgleichungen (Geschlossene Lösung)	6
1.4	Probabilistische Interpretation	7
1.5	Lokal gewichtete lineare Regression	7
2	Klassifikation und Logistische Regression	7
2.1	Logistische Regression	7
2.2	Newton-Verfahren (Fisher Scoring)	8
2.3	Mehrklassen-Klassifikation und Softmax	8
3	Generalisierte Lineare Modelle (GLMs)	8
3.1	Die Exponentialfamilie	8
3.2	GLM-Konstruktion	8
4	Generative Lernalgorithmen	9
4.1	Diskriminativ vs. Generativ	9
4.2	Gaussian Discriminant Analysis (GDA)	9
4.3	Naive Bayes	10
5	Kernel-Methoden	10
5.1	Feature Maps und das Kernel-Trick	10
5.2	Wichtige Kernels	10
5.3	Mercers Satz	11
5.4	Representer-Theorem	11
6	Support Vector Machines (SVMs)	11
6.1	Margins und Intuition	11

6.2	Primal-Problem (linear trennbar)	12
6.3	Lagrange-Dualität und Dual-Form	12
6.4	Soft-Margin SVM (nicht-trennbarer Fall)	13
6.5	SMO-Algorithmus	13
7	Perceptron und Online Learning	14
7.1	Online Learning	14
7.2	Der Perceptron-Algorithmus	14
7.3	Mistake-Bound-Theorem	14
8	Decision Trees	15
8.1	Nichtlineare, regionsbasierte Klassifikation	15
8.2	Rekursive Partitionierung	15
8.3	Verlustfunktionen für Splits	15
8.4	Regularisierung und Eigenschaften	15
9	Ensemble-Methoden	16
9.1	Varianzreduktion durch Mittelung	16
9.2	Bagging (Bootstrap Aggregation)	16
9.3	Boosting	16
9.4	Forward Stagewise Additive Modeling	17
10	Verlustfunktionen: Konvexe Ersatzverluste	18
10.1	Margin-basierte Verlustfunktionen	18
II	Deep Learning	19
11	Deep Learning	19
11.1	Nichtlineare Modelle und neuronale Netze	19
11.2	Aktivierungsfunktionen	19
11.3	Moderne Architekturbausteine	19
11.4	Backpropagation	20
11.5	Optimierungsverfahren	20
III	Generalisierung und Regularisierung	21
12	Generalisierung	21
12.1	Trainings- und Testfehler	21
12.2	Bias-Varianz-Zerlegung	21
12.3	Double Descent	23
12.4	Konzentrationsungleichungen	23
12.5	Stichprobenkomplexität	24
13	Regularisierung und Modellauswahl	24
13.1	Regularisierung	24

13.2 Bayesianische Interpretation	24
13.3 Implizite Regularisierung	25
13.4 Kreuzvalidierung (Cross-Validation)	25
14 Gaussian Processes	25
14.1 Bayessche Sichtweise auf Regression	25
14.2 Verteilungen über Funktionen	25
14.3 GP-Regression: Vorhersage	26
15 Praktische Fehleranalyse	26
15.1 Fehleranalyse (Error Analysis)	27
15.2 Ablative Analyse	27
15.3 Fehler manuell analysieren	27
15.4 Bias-Varianz-Diagnostik via Lernkurven	27
15.5 Diagnostik: Optimierungsalgorithmus vs. Optimierungsziel	28
16 Evaluationsmetriken für Klassifikation	28
16.1 Warum Genauigkeit oft nicht genügt	28
16.2 Konfusionsmatrix und Punkt-Metriken	29
16.3 Summary-Metriken: ROC- und PR-Kurven	29
IV Unüberwachtes Lernen	30
17 K-Means-Clustering	30
18 EM-Algorithmus	30
18.1 Mixture of Gaussians (MoG)	30
18.2 Jensens Ungleichung	30
18.3 Allgemeiner EM-Algorithmus	31
18.4 EM für Gauß-Gemische	31
19 Faktorenanalyse	31
19.1 Motivation: Wenn $n \gg m$	31
19.2 Das Modell	31
19.3 EM für Faktorenanalyse	32
20 Hidden Markov Models	32
20.1 Markov-Modelle	32
20.2 Hidden Markov Models: Drei Fragen	33
20.3 Forward-Prozedur (Frage 1)	33
20.4 Viterbi-Algorithmus (Frage 2)	33
20.5 Baum-Welch-Algorithmus (Frage 3)	33
21 Hauptkomponentenanalyse (PCA)	34
22 Unabhängige Komponentenanalyse (ICA)	34
22.1 ICA-Ambiguitäten	34

22.2 ICA-Algorithmus	35
23 Selbstüberwachtes Lernen und Foundation Models	35
23.1 Motivation	35
23.2 Pretraining in Computer Vision	35
23.3 Large Language Models und Transformer	35
24 Kritische Perspektiven auf ML	36
24.1 Adversarial Examples	36
24.2 Interpretierbarkeit	36
24.3 Kosten und gesellschaftliche Aspekte	37
V Bestärkendes Lernen	38
25 Bestärkendes Lernen	38
25.1 Markov-Entscheidungsprozesse	38
25.2 Bellman-Gleichungen	38
25.3 Value Iteration und Policy Iteration	38
25.4 Wertfunktions-Approximation (kontinuierliche Zustände)	39
26 Lineare Quadratische Regelung, DDP und LQG	39
26.1 Lineare Quadratische Regelung (LQR)	39
26.2 Differential Dynamic Programming (DDP)	39
26.3 Lineares Quadratisches Gauß-Modell (LQG)	39
27 Policy Gradient (REINFORCE)	40
27.1 Direkte Policy-Parametrisierung	40
27.2 Policy-Gradient-Theorem	40
27.3 REINFORCE-Algorithmus	40
27.4 Varianzreduktion	40
Schluss: Zusammenfassung der Schlüsselkonzepte	41

Vorwort

Diese Zusammenfassung begleitet die CS229-Vorlesungsreihe – basierend auf dem Skript `main_notes.pdf`, sämtlichen ergänzenden Lecture Notes im Ordner `notes/` und `extra-notes/` (Decision Trees, Ensembling, Perceptron, Faktorenanalyse, Fehleranalyse sowie John Duchis Supplemental Notes zu Hoeffdings Ungleichung, Verlustfunktionen und dem Representer-Theorem) sowie zusätzlichem Material aus `materials/` und `section/` (Andrew Ngs ML-Advice-Folien, Konvexe Optimierung/KKT, SMO-Pseudocode, Gaussian Processes, Hidden Markov Models, Evaluationsmetriken sowie kritische Perspektiven auf ML). Ziel ist es, die wesentlichen Konzepte, Algorithmen und mathematischen Grundlagen aller 27 Kapitel kompakt darzustellen – mit Herleitungen, Intuition und Querverweisen zwischen den Themen. Zwei Arten von Boxen kennzeichnen Material außerhalb des `main_notes`-Kerntexts: graue **Quelle**-Boxen verweisen auf zusätzliche, aber offizielle CS229-Materialien (Section Notes, Handouts) aus diesem Repository; orange **Zusatzinfo (Internet)**-Boxen enthalten recherchiertes Material, das nicht aus CS229-Unterlagen stammt.

Das Skript ist in fünf Teile gegliedert:

1. **Überwachtes Lernen** (Kap. 1–10): Regression, Klassifikation, generative Modelle, Kernel-Methoden, SVMs, Perceptron, Decision Trees, Ensemble-Methoden, Verlustfunktionen
2. **Deep Learning** (Kap. 11): Neuronale Netze und Backpropagation
3. **Generalisierung & Regularisierung** (Kap. 12–16): Bias-Varianz, Double Descent, Modellauswahl, Gaussian Processes, praktische Fehleranalyse, Evaluationsmetriken
4. **Unüberwachtes Lernen** (Kap. 17–24): Clustering, EM, Faktorenanalyse, Hidden Markov Models, PCA, ICA, Foundation Models, kritische Perspektiven auf ML
5. **Bestärkendes Lernen** (Kap. 25–27): MDPs, LQR, Policy Gradient

Notation. Vektoren und Matrizen werden fett geschrieben ($\mathbf{x}$, $\mathbf{W}$). Skalare kursiv (x , θ). Zufallsgrößen mit Großbuchstaben (X , Y). Trainingsdaten $\{(x^{(i)}, y^{(i)})\}_{i=1}^n$ mit n Beispielen.

Teil I

Überwachtes Lernen

1 Lineare Regression

1.1 Aufgabenstellung

Beim überwachten Lernen sei ein Datensatz $\{(x^{(i)}, y^{(i)})\}_{i=1}^n$ mit $x^{(i)} \in \mathbb{R}^d$ und $y^{(i)} \in \mathbb{R}$ gegeben. Ziel ist es, eine Hypothesenfunktion $h : \mathbb{R}^d \rightarrow \mathbb{R}$ zu lernen, die für neue Eingaben x gute Vorhersagen $h(x) \approx y$ liefert.

Bei der **linearen Regression** wird angesetzt:

$$h_{\theta}(x) = \theta^{\top} x = \sum_{j=0}^d \theta_j x_j, \quad x_0 := 1 \text{ (Bias-Term)}. \quad (1)$$

1.2 Kostenfunktion und Gradientenverfahren

Die **mittlere quadratische Abweichung** (MSE) dient als Verlustfunktion:

$$J(\theta) = \frac{1}{2n} \sum_{i=1}^n (h_{\theta}(x^{(i)}) - y^{(i)})^2. \quad (2)$$

Batch Gradient Descent minimiert J durch iterative Updates:

$$\theta_j \leftarrow \theta_j - \alpha \frac{\partial J}{\partial \theta_j} = \theta_j + \frac{\alpha}{n} \sum_{i=1}^n (y^{(i)} - h_{\theta}(x^{(i)})) x_j^{(i)}. \quad (3)$$

Stochastisches Gradientenverfahren (SGD). Bei großen Datensätzen ist ein vollständiger Batch-Update teuer. SGD wählt in jedem Schritt ein einzelnes (oder eine kleine Gruppe von) Beispiel(en) und approximiert den Gradienten:

$$\theta_j \leftarrow \theta_j + \alpha (y^{(i)} - h_{\theta}(x^{(i)})) x_j^{(i)}. \quad (4)$$

1.3 Normalgleichungen (Geschlossene Lösung)

Die **Design-Matrix** $X \in \mathbb{R}^{n \times d}$ enthält die Trainingsvektoren zeilenweise. Dann lässt sich der Verlust kompakt schreiben als

$$J(\theta) = \frac{1}{2n} \|X\theta - \mathbf{y}\|^2. \quad (5)$$

Ableiten und Null setzen ergibt die **Normalgleichungen**:

$$\boxed{\hat{\theta} = (X^{\top} X)^{-1} X^{\top} \mathbf{y}.} \quad (6)$$

Diese geschlossene Lösung ist für kleine bis mittlere d effizient; für sehr große d oder schlecht konditioniertes $X^{\top} X$ wird das iterative Verfahren bevorzugt.

1.4 Probabilistische Interpretation

Annahme: $y^{(i)} = \theta^\top x^{(i)} + \varepsilon^{(i)}$ mit iid $\varepsilon^{(i)} \sim \mathcal{N}(0, \sigma^2)$. Dann ist die Likelihood

$$L(\theta) = \prod_{i=1}^n p(y^{(i)} | x^{(i)}; \theta) = \prod_{i=1}^n \frac{1}{\sqrt{2\pi} \sigma} \exp\left(-\frac{(y^{(i)} - \theta^\top x^{(i)})^2}{2\sigma^2}\right).$$

Maximierung der Log-Likelihood ist äquivalent zur Minimierung von $J(\theta)$ – die Least-Squares-Methode ist somit der Maximum-Likelihood-Schätzer (MLE) unter der Gauß-Annahme.

1.5 Lokal gewichtete lineare Regression

Bei der **lokal gewichteten Regression** (LWLR) werden die Trainingspunkte je nach Nähe zur Anfrage x unterschiedlich gewichtet:

$$w^{(i)}(x) = \exp\left(-\frac{\|x^{(i)} - x\|^2}{2\tau^2}\right), \quad (7)$$

und der gewichtete Verlust $J(\theta) = \sum_i w^{(i)}(x)(y^{(i)} - \theta^\top x^{(i)})^2$ wird minimiert. LWLR ist ein **nichtparametrisches** Verfahren: die Lösung hängt stets von den Trainingsdaten ab.

2 Klassifikation und Logistische Regression

2.1 Logistische Regression

Bei binärer Klassifikation ($y \in \{0, 1\}$) ist ein lineares Modell ungeeignet, da $\theta^\top x \notin [0, 1]$. Stattdessen setzt man die **Sigmoid-Funktion** ein:

$$g(z) = \frac{1}{1 + e^{-z}}, \quad h_\theta(x) = g(\theta^\top x) = \frac{1}{1 + e^{-\theta^\top x}}. \quad (8)$$

Das stochastische Modell lautet:

$$P(y = 1 | x; \theta) = h_\theta(x), \quad (9)$$

$$P(y = 0 | x; \theta) = 1 - h_\theta(x). \quad (10)$$

Die Log-Likelihood über den Datensatz ist

$$\ell(\theta) = \sum_{i=1}^n \left[y^{(i)} \log h_\theta(x^{(i)}) + (1 - y^{(i)}) \log(1 - h_\theta(x^{(i)})) \right]. \quad (11)$$

Maximierung via Stochastic Gradient Ascent führt auf die Update-Regel

$$\theta_j \leftarrow \theta_j + \alpha (y^{(i)} - h_\theta(x^{(i)})) x_j^{(i)}, \quad (12)$$

die formal identisch mit der LMS-Regel aussieht – ein bemerkenswertes Ergebnis, obwohl h eine völlig andere Funktion ist.

2.2 Newton-Verfahren (Fisher Scoring)

Das Newton-Raphson-Verfahren konvergiert deutlich schneller als Gradientenverfahren, weil es Krümmungsinformation nutzt:

$$\theta \leftarrow \theta - H^{-1} \nabla_{\theta} \ell(\theta), \quad (13)$$

wobei die **Hesse-Matrix** $H_{jk} = \partial^2 \ell / (\partial \theta_j \partial \theta_k)$ für logistische Regression lautet:

$$H = -\frac{1}{n} X^{\top} S X, \quad S = \text{diag}(h_{\theta}(x^{(1)})(1 - h_{\theta}(x^{(1)})), \dots). \quad (14)$$

Newton-Schritt heißt hier auch **Fisher Scoring** und konvergiert quadratisch – ein Schritt entspricht grob $O(d)$ Gradientenschritten.

2.3 Mehrklassen-Klassifikation und Softmax

Für K Klassen parametrisiert man für jede Klasse k einen Vektor $\theta_k \in \mathbb{R}^d$ und nutzt die **Softmax**-Funktion:

$$P(y = k \mid x; \Theta) = \frac{\exp(\theta_k^{\top} x)}{\sum_{j=1}^K \exp(\theta_j^{\top} x)}. \quad (15)$$

Die zugehörige Verlustfunktion ist der **Cross-Entropy-Verlust**:

$$\ell_{\text{CE}}(\hat{y}, y) = -\log P(y \mid x; \Theta) = -\theta_y^{\top} x + \log \sum_{j=1}^K \exp(\theta_j^{\top} x). \quad (16)$$

3 Generalisierte Lineare Modelle (GLMs)

3.1 Die Exponentialfamilie

Viele Standardverteilungen lassen sich in der **Exponentialfamilien**-Form schreiben:

$$p(y; \eta) = b(y) \exp(\eta^{\top} T(y) - a(\eta)), \quad (17)$$

mit natürlichem Parameter η , hinreichender Statistik $T(y)$, Log-Partitionsfunktion $a(\eta)$ und Basismaß $b(y)$.

Bernoulli-Verteilung: $p(y; \phi) = \phi^y (1 - \phi)^{1-y}$, $y \in \{0, 1\}$. Schreibweise als Exponentialfamilie mit $\eta = \log \frac{\phi}{1-\phi}$, also $\phi = 1/(1 + e^{-\eta})$ – dies *ist* genau die Sigmoid-Funktion.

Gauß-Verteilung: $\mathcal{N}(\mu, \sigma^2)$ mit $\eta = \mu$, $T(y) = y$, $a(\eta) = \eta^2/2$, $b(y) = (2\pi)^{-1/2} e^{-y^2/2}$.

3.2 GLM-Konstruktion

Ein **Generalisiertes Lineares Modell** (GLM) basiert auf drei Annahmen:

1. $y \mid x; \theta \sim \text{ExponentialFamily}(\eta)$.
2. Hypothese: $h(x) = \mathbb{E}[T(y) \mid x; \theta]$.
3. Lineare Verbindung: $\eta = \theta^{\top} x$.

Die kanonische **Responsefunktion** $g(\eta) = \mathbb{E}[T(y); \eta]$ verbindet natürlichen Parameter und Erwartungswert. Ihre Inverse ist die **Linkfunktion**.

Verteilung	Responsefunktion	Modell	Anwendung
Gauß	Identität	Lineare Regression	Regressionsaufgaben
Bernoulli	Sigmoid	Logistische Regression	Binäre Klassifikation
Multinomial	Softmax	Softmax-Regression	Mehrklassenprobleme
Poisson	e^η	Poisson-Regression	Zählraten

4 Generative Lernalgorithmen

4.1 Diskriminativ vs. Generativ

Diskriminative Modelle (z. B. logistische Regression) modellieren $p(y | x)$ direkt. **Generative** Modelle lernen $p(x | y)$ und $p(y)$ und nutzen den Satz von Bayes:

$$p(y | x) = \frac{p(x | y) p(y)}{p(x)}, \quad p(x) = \sum_y p(x | y) p(y). \quad (18)$$

4.2 Gaussian Discriminant Analysis (GDA)

GDA modelliert $x | y$ als multivariate Normalverteilung:

$$y \sim \text{Bernoulli}(\phi), \quad (19)$$

$$x | y = 0 \sim \mathcal{N}(\mu_0, \Sigma), \quad (20)$$

$$x | y = 1 \sim \mathcal{N}(\mu_1, \Sigma). \quad (21)$$

Die **multivariate Normalverteilung** lautet:

$$p(x; \mu, \Sigma) = \frac{1}{(2\pi)^{d/2} |\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(x - \mu)^\top \Sigma^{-1}(x - \mu)\right). \quad (22)$$

Die MLE-Schätzer sind:

$$\hat{\phi} = \frac{1}{n} \sum_{i=1}^n \mathbb{1}\{y^{(i)} = 1\}, \quad (23)$$

$$\hat{\mu}_k = \frac{\sum_i \mathbb{1}\{y^{(i)} = k\} x^{(i)}}{\sum_i \mathbb{1}\{y^{(i)} = k\}}, \quad k \in \{0, 1\}, \quad (24)$$

$$\hat{\Sigma} = \frac{1}{n} \sum_{i=1}^n (x^{(i)} - \hat{\mu}_{y^{(i)}})(x^{(i)} - \hat{\mu}_{y^{(i)}})^\top. \quad (25)$$

GDA und logistische Regression. Wenn $p(x | y)$ tatsächlich Gauß-verteilt ist, hat die Posterior $p(y = 1 | x)$ stets die logistische Form $1/(1 + e^{-\theta^\top x})$. GDA macht *stärkere* Annahmen und ist asymptotisch effizienter, wenn diese zutreffen. Logistische Regression

ist robuster, wenn die Gauß-Annahme verletzt ist.

4.3 Naive Bayes

Naive Bayes setzt **bedingte Unabhängigkeit** der Features voraus:

$$p(x \mid y) = \prod_{j=1}^d p(x_j \mid y). \quad (26)$$

Dies vereinfacht die Schätzung erheblich. Für Textklassifikation mit Binärfeatures:

$$\hat{\phi}_{j|y=k} = \frac{\sum_i \mathbb{1}\{x_j^{(i)} = 1, y^{(i)} = k\}}{\sum_i \mathbb{1}\{y^{(i)} = k\}}. \quad (27)$$

Laplace-Glättung verhindert Nullwahrscheinlichkeiten für ungesehene Features:

$$\hat{\phi}_{j|y=k} = \frac{1 + \sum_i \mathbb{1}\{x_j^{(i)} = 1, y^{(i)} = k\}}{2 + \sum_i \mathbb{1}\{y^{(i)} = k\}}. \quad (28)$$

5 Kernel-Methoden

5.1 Feature Maps und das Kernel-Trick

Sei $\phi : \mathbb{R}^d \rightarrow \mathbb{R}^p$ eine Feature-Abbildung (z. B. Polynom-Features). Das LMS-Update in Feature-Raum lautet:

$$\theta \leftarrow \theta + \alpha \sum_{i=1}^n (y^{(i)} - \theta^\top \phi(x^{(i)})) \phi(x^{(i)}).$$

Da θ stets als Linearkombination der $\phi(x^{(i)})$ geschrieben werden kann ($\theta = \sum_i \beta_i \phi(x^{(i)})$), reduziert sich die Vorhersage auf Skalarprodukte:

$$h(x) = \theta^\top \phi(x) = \sum_{i=1}^n \beta_i \phi(x^{(i)})^\top \phi(x) = \sum_{i=1}^n \beta_i K(x^{(i)}, x). \quad (29)$$

Definition 5.1. Eine Funktion $K : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$ heißt **Kernel**, wenn $K(x, z) = \phi(x)^\top \phi(z)$ für eine Feature-Abbildung ϕ .

Der entscheidende Vorteil: $K(x, z)$ kann in $O(d)$ Zeit berechnet werden, obwohl $\phi(x)$ möglicherweise sehr hochdimensional (oder unendlich-dimensional) ist. Beispiel: Polynom-Kernel $K(x, z) = (x^\top z + c)^k$ entspricht einem $O(d^k)$ -dimensionalen Feature-Raum.

5.2 Wichtige Kernels

Kernel	$K(x, z)$	Feature-Raum
Linear	$x^\top z$	Original-Raum $\mathbb{R}^d$
Polynom (Grad k)	$(x^\top z + c)^k$	$O(d^k)$ -dimensional
Gauß (RBF)	$\exp(-\frac{\ x-z\ ^2}{2\sigma^2})$	∞ -dimensional

5.3 Mercers Satz

Mercers Satz. Eine Funktion $K : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$ ist genau dann ein gültiger Kernel (d. h. $K(x, z) = \phi(x)^\top \phi(z)$ für ein ϕ), wenn für beliebige Punkte $x^{(1)}, \dots, x^{(n)}$ die **Gram-Matrix** $[K]_{ij} = K(x^{(i)}, x^{(j)})$ symmetrisch und positiv semidefinit ist.

Kernels sind unter Addition, positiver Skalierung und (elementweisem) Produkt abgeschlossen – diese Rechenregeln ermöglichen den systematischen Aufbau komplexer Kernels aus einfachen.

5.4 Representer-Theorem

Warum genügt es, θ stets als Linearkombination der Trainingsbeispiele anzusetzen? Für einen regularisierten empirischen Risiko-Ansatz

$$J_\lambda(\theta) = \frac{1}{n} \sum_{i=1}^n L(\theta^\top x^{(i)}, y^{(i)}) + \frac{\lambda}{2} \|\theta\|^2, \quad (30)$$

mit einem in z konvexen Verlust $L(z, y)$ (z. B. quadratischer, logistischer oder Hinge-Verlust) und $\lambda \geq 0$ gilt:

Representer-Theorem. Es existiert ein Minimierer von J_λ , der sich als Linearkombination der Trainingsbeispiele schreiben lässt:

$$\theta = \sum_{i=1}^n \alpha_i x^{(i)} \quad (31)$$

für reelle Koeffizienten $\alpha_1, \dots, \alpha_n$.

Beweisidee (differenzierbarer Fall, $\lambda > 0$): Am Minimum verschwindet der Gradient,

$$\nabla J_\lambda(\theta) = \frac{1}{n} \sum_{i=1}^n L'(\theta^\top x^{(i)}, y^{(i)}) x^{(i)} + \lambda \theta = 0.$$

Auflösen nach θ liefert direkt $\theta = -\frac{1}{n\lambda} \sum_i L'(\theta^\top x^{(i)}, y^{(i)}) x^{(i)} =: \sum_i \alpha_i x^{(i)}$. Dies rechtfertigt im Nachhinein den Kernel-Trick aus Abschnitt 5.1: Da die Lösung stets in der von den Daten aufgespannten Linearkombination liegt, genügen zur Auswertung der Hypothese stets nur Skalarprodukte $\phi(x^{(i)})^\top \phi(x^{(j)}) = K(x^{(i)}, x^{(j)})$ – eine explizite Darstellung von θ im (ggf. unendlich-dimensionalen) Feature-Raum ist nie nötig.

6 Support Vector Machines (SVMs)

6.1 Margins und Intuition

Eine Hyperebene $\{x : w^\top x + b = 0\}$ klassifiziert Punkt $x^{(i)}$ korrekt, wenn $y^{(i)}(w^\top x^{(i)} + b) > 0$. Der **funktionale Margin** misst diese Konfidenz:

$$\hat{\gamma}^{(i)} = y^{(i)}(w^\top x^{(i)} + b), \quad (32)$$

und der **geometrische Margin** (abstands basiert) ist

$$\gamma^{(i)} = \frac{y^{(i)}(w^\top x^{(i)} + b)}{\|w\|}. \quad (33)$$

Das SVM-Ziel: finde die Hyperebene, die alle Punkte korrekt klassifiziert und deren *minimaler geometrischer Margin* maximal ist.

6.2 Primal-Problem (linear trennbar)

Mit der Normierung $\min_i \hat{\gamma}^{(i)} = 1$ wird das **Optimale-Margin-Klassifikatorproblem**:

$$\min_{w,b} \frac{1}{2} \|w\|^2 \quad \text{s.t.} \quad y^{(i)}(w^\top x^{(i)} + b) \geq 1, \quad i = 1, \dots, n. \quad (34)$$

Dies ist ein konvexes quadratisches Programm.

6.3 Lagrange-Dualität und Dual-Form

Der Lagrangian ist

$$\mathcal{L}(w, b, \alpha) = \frac{1}{2} \|w\|^2 - \sum_{i=1}^n \alpha_i [y^{(i)}(w^\top x^{(i)} + b) - 1].$$

Aus den KKT-Stationaritätsbedingungen folgt $w = \sum_i \alpha_i y^{(i)} x^{(i)}$. Einsetzen ergibt das **Dual-Problem**:

$$\max_{\alpha} W(\alpha) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y^{(i)} y^{(j)} \langle x^{(i)}, x^{(j)} \rangle \quad (35)$$

s.t. $\alpha_i \geq 0$ und $\sum_i \alpha_i y^{(i)} = 0$.

Die Vorhersage hängt damit nur noch von Skalarprodukten ab – und lässt sich mit einem Kernel verallgemeinern: $\langle x^{(i)}, x^{(j)} \rangle \rightarrow K(x^{(i)}, x^{(j)})$.

Support Vectors sind die Punkte mit $\alpha_i > 0$; nur sie definieren die Entscheidungsgrenze.

Die "KKT-Stationaritätsbedingungen", die oben zur Herleitung von w genutzt wurden, sind ein Spezialfall eines allgemeinen Satzes der konvexen Optimierung, der Optimalität für *jedes* Primal-Dual-Paar charakterisiert:

Karush-Kuhn-Tucker (KKT)-Theorem. Für ein konvexes Problem $\min_x f(x)$ s.t. $g_i(x) \leq 0$ ($i = 1, \dots, m$), $h_i(x) = 0$ ($i = 1, \dots, p$) mit Lagrangian $\mathcal{L}(x, \alpha, \beta) = f(x) + \sum_i \alpha_i g_i(x) + \sum_i \beta_i h_i(x)$ gilt: x^* ist primal-optimal und (α^*, β^*) dual-optimal genau dann, wenn (bei starker Dualität) die folgenden vier Bedingungen erfüllt sind:

1. **Primale Zulässigkeit:** $g_i(x^*) \leq 0$, $h_i(x^*) = 0$.
2. **Duale Zulässigkeit:** $\alpha_i^* \geq 0$.
3. **Komplementärer Schlupf (complementary slackness):** $\alpha_i^* g_i(x^*) = 0$.
4. **Stationarität:** $\nabla_x \mathcal{L}(x^*, \alpha^*, \beta^*) = 0$.

Bedingung 3 erklärt direkt, warum nur Support Vectors ($\alpha_i > 0$) die Lösung bestimmen: Für alle anderen Punkte muss die zugehörige Ungleichungsnebenbedingung $g_i(x^*) = 0$ sein (d. h. $y^{(i)}(w^\top x^{(i)} + b) = 1$, der Punkt liegt exakt auf dem Margin) oder $\alpha_i^* = 0$ (der Punkt trägt nicht zur Lösung bei).

Quelle

Quelle: Convex Optimization Overview (cnt'd), Chuong B. Do (CS229 Section Notes) – [section/cs229-cvxopt2.pdf](#), Abschnitt 1.6.

6.4 Soft-Margin SVM (nicht-trennbarer Fall)

Wenn die Daten nicht linear trennbar sind, erlaubt man Schlupfvariablen $\xi_i \geq 0$ (Verletzungen der Margin-Bedingung):

$$\min_{w,b,\xi} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i \quad \text{s.t.} \quad y^{(i)}(w^\top x^{(i)} + b) \geq 1 - \xi_i, \quad \xi_i \geq 0. \quad (36)$$

Der Hyperparameter $C > 0$ steuert den Trade-off zwischen großem Margin und kleinen Fehlern. Das Dual-Problem ändert sich nur in $0 \leq \alpha_i \leq C$.

6.5 SMO-Algorithmus

Der **Sequential Minimal Optimization (SMO)** Algorithmus löst das quadratische Programm, indem er iterativ immer zwei α_i simultan aktualisiert (da die Gleichheitsbedingung $\sum_i \alpha_i y^{(i)} = 0$ ein einzelnes Update verbietet). Die Konvergenz wird durch Prüfung der KKT-Bedingungen überwacht. Für die Soft-Margin-SVM lauten die KKT-Bedingungen konkret:

$$\alpha_i = 0 \Rightarrow y^{(i)}(w^\top x^{(i)} + b) \geq 1, \quad \alpha_i = C \Rightarrow y^{(i)}(w^\top x^{(i)} + b) \leq 1, \quad 0 < \alpha_i < C \Rightarrow y^{(i)}(w^\top x^{(i)} + b) = 1. \quad (37)$$

Vereinfachtes SMO (wie in Aufgabenstellungen zu Problem Set 2 verwendet): Statt einer aufwendigen Heuristik zur Auswahl des Paares (α_i, α_j) iteriert man einfach über alle α_i ; verletzt α_i die KKT-Bedingungen, wird ein zufälliges α_j gewählt und beide gemeinsam optimiert:

Update-Schritt für ein Paar (α_i, α_j)

1. Grenzen bestimmen (damit $0 \leq \alpha_j \leq C$ erhalten bleibt):

$$y^{(i)} \neq y^{(j)} : L = \max(0, \alpha_j - \alpha_i), \quad H = \min(C, C + \alpha_j - \alpha_i) \quad y^{(i)} = y^{(j)} : L = \max(0, \alpha_i + \alpha_j - C), \quad H = \min(C, \alpha_i + \alpha_j)$$

2. Unbeschränktes Optimum und Clipping mit $E_k = f(x^{(k)}) - y^{(k)}$ (Vorhersagefehler) und $\eta = 2K(x^{(i)}, x^{(j)}) - K(x^{(i)}, x^{(i)}) - K(x^{(j)}, x^{(j)})$:

$$\alpha_j \leftarrow \alpha_j - \frac{y^{(j)}(E_i - E_j)}{\eta}, \quad \alpha_j \leftarrow \text{clip}(\alpha_j, L, H).$$

3. Update von α_i : $\alpha_i \leftarrow \alpha_i + y^{(i)}y^{(j)}(\alpha_j^{\text{alt}} - \alpha_j)$.

4. Bias b neu setzen so, dass die KKT-Bedingungen für i, j erfüllt bleiben (Mittelwert der beiden Randfälle, falls beide α im Inneren $(0, C)$ liegen).

Quelle

Quelle: CS229 Simplified SMO Algorithm, CS229 Autumn 2009 (basierend auf J. Platts Originalarbeit) – *materials/smo.pdf*.

7 Perceptron und Online Learning

7.1 Online Learning

Im Gegensatz zum **Batch Learning** (Trainingsdaten vollständig vorab verfügbar) erhält der Lernalgorithmus beim **Online Learning** eine Sequenz von Beispielen $(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots$ nacheinander: für jedes $x^{(i)}$ wird zunächst eine Vorhersage getroffen, bevor das wahre Label $y^{(i)}$ offengelegt wird (und ggf. für weiteres Lernen genutzt werden kann). Von Interesse ist die **Gesamtzahl der Fehler**, die der Algorithmus während dieses Prozesses macht.

7.2 Der Perceptron-Algorithmus

Mit Labels $y \in \{-1, +1\}$ und Parametern $\theta \in \mathbb{R}^{d+1}$ lautet die Hypothese

$$h_{\theta}(x) = g(\theta^{\top} x), \quad g(z) = \begin{cases} 1 & z \geq 0 \\ -1 & z < 0. \end{cases} \quad (38)$$

Bei korrekter Klassifikation bleibt θ unverändert; bei einem Fehler ($h_{\theta}(x) \neq y$ für Beispiel (x, y)) erfolgt das Update

$$\theta \leftarrow \theta + y x. \quad (39)$$

Anders als bei LMS oder logistischer Regression modelliert der Perceptron keine Wahrscheinlichkeit – er ist ein rein diskriminatives Entscheidungsverfahren, das nur bei Fehlern reagiert.

7.3 Mistake-Bound-Theorem

Satz (Block 1962, Novikoff 1962). Sei eine Sequenz $(x^{(1)}, y^{(1)}), \dots, (x^{(m)}, y^{(m)})$ gegeben mit $\|x^{(i)}\| \leq D$ für alle i , und es existiere ein Einheitsvektor u ($\|u\| = 1$) mit $y^{(i)}(u^{\top} x^{(i)}) \geq \gamma$ für alle i (d. h. u trennt die Daten mit Margin γ). Dann macht der Perceptron-Algorithmus höchstens $(D/\gamma)^2$ Fehler auf dieser Sequenz.

Bemerkenswert: Die Schranke hängt *weder* von der Anzahl der Beispiele m noch von der Dimension d ab – nur vom (normierten) Margin γ und dem Datenradius D . **Beweisidee:** Sei $\theta^{(k)}$ der Parameter beim k -ten Fehler ($\theta^{(1)} = \vec{0}$). Einerseits wächst $(\theta^{(k+1)})^{\top} u \geq k\gamma$ (jedes Update erhöht die Projektion auf u um mind. γ), andererseits wächst $\|\theta^{(k+1)}\|^2 \leq kD^2$ (da bei einem Fehler $y^{(i)}(x^{(i)})^{\top} \theta^{(k)} \leq 0$ gilt, folgt $\|\theta^{(k+1)}\|^2 \leq \|\theta^{(k)}\|^2 + D^2$). Mit Cauchy-Schwarz ($\|\theta^{(k+1)}\| \geq (\theta^{(k+1)})^{\top} u$, da u Einheitsvektor) folgt $\sqrt{k}D \geq k\gamma$, also $k \leq (D/\gamma)^2$.

8 Decision Trees

8.1 Nichtlineare, regionsbasierte Klassifikation

Decision Trees sind die ersten genuin **nichtlinearen** Verfahren in dieser Zusammenfassung: Sie partitionieren den Eingaberaum $\mathcal{X}$ rekursiv in disjunkte Regionen, $\mathcal{X} = \bigcup_i R_i$ mit $R_i \cap R_j = \emptyset$ für $i \neq j$, und sagen pro Region einen konstanten Wert voraus (Mehrheitsklasse bei Klassifikation, Mittelwert bei Regression).

8.2 Rekursive Partitionierung

Da optimale Regionen im Allgemeinen inaktuell zu bestimmen sind, wählen Decision Trees einen **gierigen, Top-Down-Ansatz**: Gegeben eine Elternregion R_p , Feature-Index j und Schwellwert t :

$$R_1 = \{x \in R_p : x_j < t\}, \quad R_2 = \{x \in R_p : x_j \geq t\}. \quad (40)$$

Bei jedem Schritt werden Blattregion, Feature und Schwelle so gewählt, dass der *Rückgang* des kardinalitätsgewichteten Verlusts maximal wird:

$$\max_{R_p, j, t} L(R_p) - \frac{|R_1| L(R_1) + |R_2| L(R_2)}{|R_1| + |R_2|}. \quad (41)$$

8.3 Verlustfunktionen für Splits

Für Klassifikation mit Klassenanteil $\hat{p}_c$ (Anteil der Beispiele der Klasse c) in Region R :

$$\text{Missklassifikationsverlust: } L_{\text{misclass}}(R) = 1 - \max_c \hat{p}_c, \quad (42)$$

$$\text{Cross-Entropy: } L_{\text{cross}}(R) = - \sum_c \hat{p}_c \log_2 \hat{p}_c. \quad (43)$$

Warum Cross-Entropy statt Missklassifikationsverlust? Der Missklassifikationsverlust ist nicht *strikt konkav* – ein Split kann daher den gewichteten Verlust exakt unverändert lassen, obwohl er informativ ist (Beispiel: Elternregion 400+/100–; Split A in 150+/100– und 250+/0– hat denselben gewichteten Missklassifikationsverlust wie ein uninformativer Split B). Cross-Entropy (und Gini-Loss) sind strikt konkav: Solange $\hat{p}_1 \neq \hat{p}_2$ in nicht-leeren Kindern, sinkt der gewichtete Verlust garantiert gegenüber dem Elternknoten. Der Rückgang heißt **Information Gain**.

Für Regression wird pro Blatt der Mittelwert $\hat{y} = \frac{1}{|R|} \sum_{i \in R} y_i$ vorhergesagt, und der quadratische Verlust $L_{\text{squared}}(R) = \frac{1}{|R|} \sum_{i \in R} (y_i - \hat{y})^2$ steuert die Split-Wahl.

8.4 Regularisierung und Eigenschaften

Ein voll ausgewachsener Baum (ein Trainingspunkt pro Blatt) hat niedrigen Bias, aber hohe Varianz. Übliche Stop- und Pruning-Heuristiken:

- Mindestgröße pro Blatt, maximale Tiefe, maximale Blattzahl.

- **Pruning:** Baum zunächst voll wachsen lassen, dann Knoten entfernen, die den Validierungsfehler minimal erhöhen – robuster als ein Mindest-Loss-Rückgang pro Split, da sonst Interaktionseffekte höherer Ordnung (die erst nach mehreren Splits sichtbar werden) übersehen würden.

Laufzeit: $O(d)$ zur Vorhersage (Baumtiefe d), $O(nfd)$ zum Training (n Beispiele, f Features).

Vorteile: interpretierbar, verarbeitet kategoriale Features nativ (Splits über Teilmengen), schnell.

Nachteile: hohe Varianz, kann additive Struktur (z. B. eine Grenze der Form $x_1 + x_2 = c$) nur durch viele Splits approximieren, obwohl ein lineares Modell sie direkt abbilden könnte.

9 Ensemble-Methoden

9.1 Varianzreduktion durch Mittelung

Für n identisch verteilte, paarweise korrelierte Zufallsvariablen (z. B. die Fehler einzelner Modelle) mit Varianz σ^2 und Korrelation ρ gilt für den Mittelwert:

$$\text{Var}(\bar{X}) = \rho\sigma^2 + \frac{1-\rho}{n}\sigma^2. \quad (44)$$

Mehr Modelle ($n \uparrow$) lässt den zweiten Term verschwinden; geringere Korrelation zwischen den Modellen ($\rho \downarrow$) senkt beide Terme. Ensembling zielt daher darauf ab, **viele, wenig korrelierte** Modelle zu kombinieren.

9.2 Bagging (Bootstrap Aggregation)

Bootstrap: Ziehe aus dem Trainingsset S ($|S| = n$) wiederholt M Stichproben $Z_1, \dots, Z_M$ mit Zurücklegen (je Größe n). Trainiere je ein Modell G_m auf Z_m und middle:

$$G(x) = \frac{1}{M} \sum_{m=1}^M G_m(x). \quad (45)$$

Bagging senkt ρ (Modelle sehen unterschiedliche Stichproben) und damit die Varianz; der Bias steigt leicht (jede Bootstrap-Stichprobe enthält im Schnitt nur $\approx 63\%$ der eindeutigen Trainingspunkte), was in der Praxis meist durch den Varianzgewinn aufgewogen wird. Mehr Modelle können *nicht* zu mehr Overfitting führen, da ρ unabhängig von M ist. **Out-of-Bag-Schätzung:** Das im Schnitt $\approx 1/3$ nicht gezogene Restdrittel jeder Stichprobe dient als kostenlose Validierungsmenge – im Grenzwert $M \rightarrow \infty$ äquivalent zu Leave-One-Out Cross-Validation.

Random Forests. Bagging von Decision Trees, wobei an jedem Split zusätzlich nur eine *zufällige Teilmenge* der Features zur Auswahl steht. Dies dekorreliert die Bäume weiter (ein einzelner starker Prädiktor dominiert nicht mehr jeden Baum) und erlaubt robusteren Umgang mit fehlenden Features, da nicht jeder Baum von jedem Feature abhängt.

9.3 Boosting

Während Bagging Varianz reduziert, reduziert **Boosting** den **Bias**: Es kombiniert viele **schwache Lerner** (hoher Bias, niedrige Varianz – z. B. Decision Stumps, Bäume der Tiefe 1) sequenti-

ell, wobei jeder neue Lerner sich verstärkt auf die Fehler der bisherigen Ensemble-Vorhersage konzentriert.

AdaBoost

1. Initialisiere Gewichte $w_i \leftarrow 1/N$ für alle $i = 1, \dots, N$.
2. Für $m = 1, \dots, M$:
 - (a) Trainiere schwachen Klassifikator G_m auf den gewichteten Daten.
 - (b) Gewichteter Fehler: $\text{err}_m = \frac{\sum_i w_i \mathbb{1}\{y_i \neq G_m(x_i)\}}{\sum_i w_i}$.
 - (c) Klassifikatorgewicht: $\alpha_m = \log\left(\frac{1-\text{err}_m}{\text{err}_m}\right)$.
 - (d) Update: $w_i \leftarrow w_i \exp(\alpha_m \mathbb{1}\{y_i \neq G_m(x_i)\})$.
3. Ausgabe: $f(x) = \text{sign}(\sum_{m=1}^M \alpha_m G_m(x))$.

Falsch klassifizierte Beispiele erhalten höheres Gewicht – nachfolgende Lerner konzentrieren sich auf die "schwierigen" Fälle. Da spätere Lerner von früheren abhängen (im Gegensatz zur Unabhängigkeit bei Bagging), steigt mit wachsendem M das Overfitting-Risiko.

9.4 Forward Stagewise Additive Modeling

AdaBoost ist ein Spezialfall des allgemeineren Schemas: Baue $f_m(x) = f_{m-1}(x) + \beta_m G(x; \gamma_m)$ iterativ auf, wobei bei jedem Schritt

$$(\beta_m, \gamma_m) = \arg \min_{\beta, \gamma} \sum_{i=1}^N L(y_i, f_{m-1}(x_i) + \beta G(x_i; \gamma)) \quad (46)$$

gelöst wird. Für quadratischen Verlust entspricht dies dem Fitten auf die Residuen $y_i - f_{m-1}(x_i)$; für binäre Klassifikation mit **exponentiellem Verlust** $L(y, \hat{y}) = \exp(-y\hat{y})$ ergibt sich exakt AdaBoost. **Gradient Boosting** verallgemeinert dies auf beliebige differenzierbare Verluste: Bei jedem Schritt wird ein neuer schwacher Lerner an den negativen Gradienten $g_i = -\partial L(y_i, f(x_i))/\partial f(x_i)$ angepasst (Regression von $G(x_i; \gamma)$ auf g_i per Least Squares).

Zusatzinfo (Internet)

Moderne Gradient-Boosting-Implementierungen – **XGBoost** (Chen & Guestrin, 2016), **LightGBM** und **CatBoost** – dominieren bis heute Kaggle-Wettbewerbe und produktive ML-Systeme auf strukturierten/ tabellarischen Daten (Betrugserkennung, Kreditrisiko, Nachfrageprognosen); laut mehreren Quellen stecken sie hinter der Mehrzahl der Gewinnerlösungen für tabellarische Aufgaben. Sie erweitern das hier vorgestellte Gradient-Boosting-Schema um explizite Regularisierung, effizientes histogrammbasiertes Splitting sowie native Unterstützung fehlender bzw. kategorialer Werte. Random Forests (Breiman, 2001) selbst bauen auf Bagging (Breiman, 1996) und zufälliger Feature-Auswahl bei Splits (Ho, 1995/1998; Amit & Geman, 1997) auf.

10 Verlustfunktionen: Konvexe Ersatzverluste

10.1 Margin-basierte Verlustfunktionen

Für binäre Klassifikation mit $y \in \{-1, +1\}$ heißt $z = y\theta^\top x$ der **Margin** eines Beispiels: $z > 0$ bedeutet korrekte Klassifikation. Der naheliegende Zielverlust wäre der **0/1-Verlust** $\phi_{01}(z) = \mathbb{1}\{z \leq 0\}$ – er ist jedoch unstetig, nicht konvex und NP-hart zu minimieren. Man verwendet daher **konvexe Ersatzverluste** (surrogate losses) $\phi(z)$ mit $\phi(z) \rightarrow 0$ für $z \rightarrow \infty$ und $\phi(z) \rightarrow \infty$ für $z \rightarrow -\infty$:

Verlust	$\phi(z)$	Zugehörige Methode
Logistisch	$\log(1 + e^{-z})$	Logistische Regression (Abschnitt 2)
Hinge	$\max(1 - z, 0)$	Support Vector Machine (Abschnitt 6)
Exponentiell	e^{-z}	AdaBoost (Abschnitt 9)

Verbindendes Prinzip. Logistische Regression, SVMs und Boosting lösen scheinbar verschiedene Probleme – tatsächlich minimieren alle drei Verfahren $\frac{1}{n} \sum_i \phi(y^{(i)}\theta^\top x^{(i)})$ für dieselbe Größe (den Margin), nur mit unterschiedlichem ϕ . Alle drei Verluste sind konvexe obere Schranken an den 0/1-Verlust und daher effizient optimierbar, während sie dessen Zielrichtung (Vorzeichen des Margins) approximieren.

Teil II

Deep Learning

11 Deep Learning

11.1 Nichtlineare Modelle und neuronale Netze

Lineare Modelle haben beschränkte Ausdruckskraft. Durch Komposition nichtlinearer Schichten kann ein neuronales Netz beliebig komplexe Funktionen approximieren. Ein **Fully-Connected Netz** mit r Schichten ist rekursiv definiert:

$$a^{[0]} = x, \quad (47)$$

$$z^{[\ell]} = W^{[\ell]} a^{[\ell-1]} + b^{[\ell]}, \quad (48)$$

$$a^{[\ell]} = \sigma(z^{[\ell]}), \quad \ell = 1, \dots, r-1, \quad (49)$$

$$\hat{y} = W^{[r]} a^{[r-1]} + b^{[r]}. \quad (50)$$

Ohne nichtlineare Aktivierungsfunktion σ kollabiert das Netz zu einem linearen Modell: $W^{[r]} \dots W^{[1]} x = \tilde{W}x$.

11.2 Aktivierungsfunktionen

Name	Formel	Typischer Einsatz
Sigmoid	$\sigma(z) = 1/(1 + e^{-z})$	Ausgabeschicht (binäre Klassifikation)
Tanh	$(e^z - e^{-z})/(e^z + e^{-z})$	Veraltete versteckte Schichten
ReLU	$\max(z, 0)$	Versteckte Schichten (Standard)
Leaky ReLU	$\max(z, \gamma z), \gamma \in (0, 1)$	Verhindert "Dying ReLU"
GELU	$z \cdot \Phi(z)$	Transformer-Modelle
Softmax	$e^{z_k} / \sum_j e^{z_j}$	Ausgabeschicht (Mehrklassen)

11.3 Moderne Architekturbausteine

Residualverbindungen (ResNets)

Um den **Vanishing-Gradient**-Effekt in tiefen Netzen zu bekämpfen, addiert ResNet den Eingabewert direkt zur Ausgabe eines Blocks:

$$\text{Res}(z) = z + \sigma(W_2 \sigma(W_1 z + b_1) + b_2). \quad (51)$$

Der Gradient kann so ungehindert durch viele Schichten fließen.

Layer-Normalisierung

$$\text{LN}(z) = \gamma \odot \frac{z - \hat{\mu}}{\hat{\sigma}} + \beta, \quad (52)$$

wobei $\hat{\mu}, \hat{\sigma}$ Mittelwert und Standardabweichung der Einträge von z sind. Wichtige Eigenschaft: Skaleninvarianz bezüglich der Gewichtsmatrizen.

Faltungsschichten (CNNs)

Eine 1D-Faltungsschicht mit Filter $w \in \mathbb{R}^k$ und Stride s berechnet:

$$\text{Conv}(z)_i = \sum_{j=1}^k w_j z_{i \cdot s - \ell + (j-1)}. \quad (53)$$

Parameter-Sharing reduziert die Parameterzahl von $O(m^2)$ auf $O(km)$.

11.4 Backpropagation

Theorem (Backpropagation). Lässt sich ein Rechenausdruck der Größe N für den Verlust in einem Forward Pass auswerten, so lässt sich der vollständige Gradient $\nabla_{\theta} J$ ebenfalls in $O(N)$ Zeit berechnen.

Algorithmisch. Sei der Forward Pass:

$$u^{[0]} = x, \quad u^{[\ell]} = M_{\ell}(u^{[\ell-1]}; \theta^{[\ell]}), \quad J = u^{[L]}.$$

Der Backward Pass berechnet rückwärts:

$$\frac{\partial J}{\partial u^{[L]}} = 1, \quad (54)$$

$$\frac{\partial J}{\partial u^{[\ell-1]}} = \underbrace{B[M_{\ell}, u^{[\ell-1]}]}_{\text{Backward-Funktion}} \left(\frac{\partial J}{\partial u^{[\ell]}} \right), \quad (55)$$

$$\frac{\partial J}{\partial \theta^{[\ell]}} = B_{\theta}[M_{\ell}, \theta^{[\ell]}] \left(\frac{\partial J}{\partial u^{[\ell]}} \right). \quad (56)$$

11.5 Optimierungsverfahren

Mini-Batch SGD

1. Initialisiere θ (z. B. zufällig)
2. Wiederhole bis Konvergenz:
 - (a) Sample Batch $\mathcal{B} \subset \{1, \dots, n\}$, $|\mathcal{B}| = B$
 - (b) $g \leftarrow \frac{1}{B} \sum_{i \in \mathcal{B}} \nabla_{\theta} J^{(i)}(\theta)$
 - (c) $\theta \leftarrow \theta - \alpha g$

Teil III

Generalisierung und Regularisierung

12 Generalisierung

12.1 Trainings- und Testfehler

Das eigentliche Ziel beim maschinellen Lernen ist die Minimierung des **Testfehlers** (auch: generalization error):

$$L(\theta) = \mathbb{E}_{(x,y) \sim \mathcal{D}} [(y - h_{\theta}(x))^2]. \quad (57)$$

Der **Trainingsfehler** $J(\theta) = \frac{1}{n} \sum_i (y^{(i)} - h_{\theta}(x^{(i)}))^2$ ist lediglich eine Approximation.

- **Overfitting:** $J(\theta) \ll L(\theta)$ – Modell zu komplex.
- **Underfitting:** $J(\theta) \approx L(\theta)$ groß – Modell zu einfach.

12.2 Bias-Varianz-Zerlegung

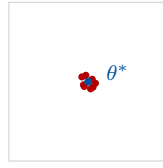
Intuition: Was bedeuten Bias und Varianz?

Betrachtet man denselben Datensatz $\{(x^{(i)}, y^{(i)})\}$ und passt sowohl ein lineares als auch ein Grad-5-Polynom an, zeigt sich ein charakteristischer Unterschied im Fehlverhalten: Das lineare Modell liegt *systematisch* daneben – selbst mit beliebig viel Trainingsdaten würde eine Gerade die zugrunde liegende (nichtlineare) Struktur nicht erfassen können. Das Grad-5-Polynom hingegen trifft die Trainingspunkte fast perfekt, aber die konkrete Kurvenform hängt stark davon ab, *welche* zufällige Stichprobe man gerade gezogen hat – ein neuer, gleich großer Datensatz aus derselben Verteilung würde zu einer sichtbar anderen Kurve führen.

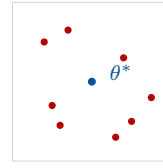
- **Bias** = der Fehler, der *übrig bliebe, selbst wenn man unendlich viele Trainingsdaten hätte*. Er entsteht rein dadurch, dass die gewählte Hypothesenklasse (z. B. "alle Geraden") die wahre Funktion h^* grundsätzlich nicht abbilden kann – ein Ausdrucksstärke-Problem, kein Datenmengenproblem. Mehr Daten helfen hier *nicht*.
- **Varianz** = wie stark das gelernte Modell h_S schwankt, wenn man den Trainingsdatensatz S (fester Größe) wiederholt neu zieht. Sie entsteht dadurch, dass ein zu flexibles Modell "zufällige" Muster im jeweiligen Rauschen $\xi^{(i)}$ mitlernt, statt nur das eigentliche Signal. Mehr Daten *reduzieren* die Varianz, da einzelne Rauschschläge im Mittel weniger Gewicht bekommen.

Im Parameterraum betrachtet

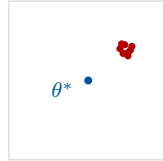
Dieselbe Intuition lässt sich direkt im Parameterraum θ visualisieren. Sei θ^* der Parameter des *bestmöglichen* Modells innerhalb der gewählten Hypothesenklasse (also der Grenzwert von $\hat{\theta}_S$ bei unendlich großem Trainingsset). Zieht man stattdessen k verschiedene, endlich große Trainingssets $S_1, \dots, S_k$ derselben Größe und trainiert jeweils separat, erhält man k Schätzungen $\hat{\theta}_{S_1}, \dots, \hat{\theta}_{S_k}$ – eine "Punktwolke" um θ^* :



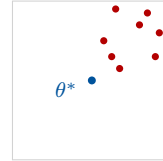
niedriger Bias, niedrige Varianz



niedriger Bias, hohe Varianz



hoher Bias, niedrige Varianz



hoher Bias, hohe Varianz

Jeder rote Punkt ist die Parameterschätzung $\hat{\theta}_S$ aus *einem* zufällig gezogenen Trainingsdatensatz; der blaue Punkt markiert θ^* . **Bias** ist der Abstand zwischen θ^* und dem *Zentrum* der Punktwolke ($\mathbb{E}_S[\hat{\theta}_S]$) – eine systematische Verschiebung, die auch bei unendlich vielen Wiederholungen bestehen bliebe. **Varianz** ist die *Streuung* der Punktwolke um ihr eigenes Zentrum – unabhängig davon, ob dieses Zentrum überhaupt nahe an θ^* liegt.

Die Verbindung zur Modellkomplexität: Ein **einfaches Modell** (z. B. lineare Regression) hat einen kleinen, stark eingeschränkten Parameterraum – unterschiedliche Trainingssets führen daher zu sehr ähnlichen Schätzungen $\hat{\theta}_S$ (**niedrige Varianz**), aber falls die wahre Funktion außerhalb dieses eingeschränkten Raums liegt, sitzt auch das Zentrum der Punktwolke systematisch daneben (**hoher Bias**) – typischerweise der untere linke Fall oben. Ein **komplexes Modell** (z. B. Grad-5-Polynomial) hat einen großen, flexiblen Parameterraum, der θ^* im Mittel gut treffen kann (**niedriger Bias**), aber jede einzelne Schätzung "jagt" das Rauschen der jeweiligen Stichprobe und landet daher an sehr unterschiedlichen Stellen (**hohe Varianz**) – der obere rechte Fall.

Formale Zerlegung

Für das Regressionsproblem lässt sich der erwartete MSE an einem Punkt x zerlegen:

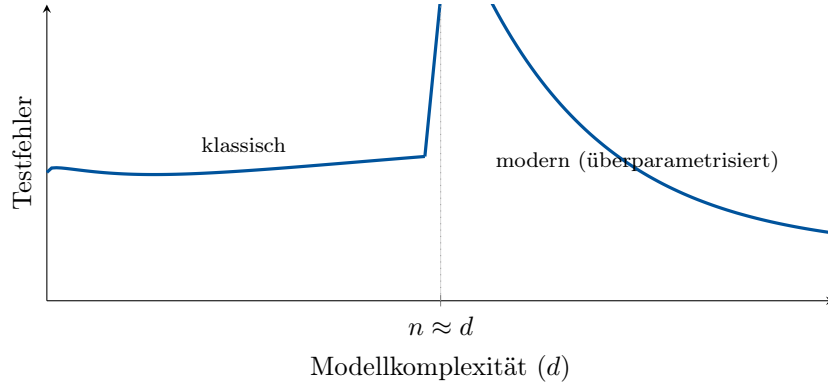
$$\mathbb{E}[(y - h_S(x))^2] = \underbrace{\sigma^2}_{\text{irreduzibles Rauschen}} + \underbrace{(h^*(x) - \bar{h}(x))^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}[(\bar{h}(x) - h_S(x))^2]}_{\text{Varianz}}, \quad (58)$$

wobei h^* die wahre Funktion, $\bar{h} = \mathbb{E}_S[h_S]$ der Erwartungswert des gelernten Modells über zufällige Trainingsdatensätze S ist – im Parameterraum-Bild oben entspricht $\bar{h}$ also gerade dem Zentrum der Punktwolke.

- Einfache Modelle (h wenig ausdrucksstark) $\Rightarrow$ hoher Bias.
- Komplexe Modelle (viele Parameter) $\Rightarrow$ hohe Varianz.
- Optimale Modellkomplexität balanciert beides.

12.3 Double Descent

Das klassische Bild (Testfehler fällt, dann steigt er bei Überanpassung) gilt nur im *classical regime* ($n \gg d$). Im modernen Deep-Learning-Regime $d \gg n$ tritt ein zweites Absinken auf:



Der Peak tritt auf, wenn $n \approx d$ – das Modell interpoliert die Trainingsdaten, verhält sich aber schlecht bei unregelmäßiger Interpolation. Im überparametrisierten Regime wählt Gradient Descent implizit die *Minimum-Norm-Lösung*, die generell gut generalisiert.

12.4 Konzentrationsungleichungen

Grundlage der Stichprobenkomplexität ist die Frage: Wie nah liegt der Trainingsfehler $J(\theta)$ am wahren Testfehler $L(\theta)$? Antwort liefern **Konzentrationsungleichungen** – eine Kette zunehmend schärferer Abschätzungen (Markov $\rightarrow$ Chebyshev $\rightarrow$ Chernoff), deren wichtigster Spezialfall für beschränkte Zufallsvariablen die Hoeffdingsche Ungleichung ist.

Hoeffdingsche Ungleichung. Seien $Z_1, \dots, Z_n$ unabhängige, beschränkte Zufallsvariablen mit $Z_i \in [a, b]$. Dann gilt für alle $t \geq 0$:

$$P\left(\left|\frac{1}{n} \sum_{i=1}^n (Z_i - \mathbb{E}[Z_i])\right| \geq t\right) \leq 2 \exp\left(-\frac{2nt^2}{(b-a)^2}\right). \quad (59)$$

Für den Trainingsfehler einer *festen* Hypothese h mit $Z_i = \mathbb{1}\{h(x^{(i)}) \neq y^{(i)}\} \in \{0, 1\}$ liefert dies ($a = 0, b = 1$):

$$P(|J(h) - L(h)| \geq t) \leq 2 \exp(-2nt^2). \quad (60)$$

Diese Schranke gilt jedoch nur für eine im Voraus fixierte Hypothese. Da $\hat{h}$ erst *nach* Betrachten der Trainingsdaten aus $\mathcal{H}$ ausgewählt wird, benötigt man eine **gleichmäßige Konvergenz** ("uniform convergence") über *alle* $h \in \mathcal{H}$ simultan – per Union Bound über $|\mathcal{H}| = k$ Hypothesen ergibt sich (Auflösen von $\delta = 2k \exp(-2nt^2)$ nach t) genau die Schranke der folgenden Stichprobenkomplexität.

12.5 Stichprobenkomplexität

Für eine endliche Hypothesenklasse $\mathcal{H}$ mit $|\mathcal{H}| = k$ gilt mit Wahrscheinlichkeit $\geq 1 - \delta$:

$$L(\hat{\theta}) \leq J(\hat{\theta}) + \sqrt{\frac{\log(k/\delta)}{2n}}. \quad (61)$$

Die Generalisierungslücke schrumpft mit $1/\sqrt{n}$ und wächst logarithmisch mit $|\mathcal{H}|$. Für unendliche $\mathcal{H}$ (z. B. alle linearen Klassifikatoren) übernimmt die **VC-Dimension** die Rolle von $\log |\mathcal{H}|$.

VC-Dimension (Vapnik–Chervonenkis). Die VC-Dimension $VC(\mathcal{H})$ einer Hypothesenklasse ist die Größe der größten Punktmenge, die $\mathcal{H}$ in *jeder* möglichen Beschriftung korrekt klassifizieren kann ("shattern"). Für lineare Klassifikatoren in $\mathbb{R}^d$ gilt $VC(\mathcal{H}) = d + 1$. Mit Wahrscheinlichkeit $\geq 1 - \delta$ gilt dann für *alle* $h \in \mathcal{H}$ gleichzeitig:

$$L(h) \leq J(h) + O\left(\sqrt{\frac{VC(\mathcal{H})}{n} \log \frac{n}{VC(\mathcal{H})}} + \frac{1}{n} \log \frac{1}{\delta}\right). \quad (62)$$

Die für eine ε -genaue Schätzung benötigte Stichprobengröße wächst linear mit $VC(\mathcal{H})$ – bei linearen Modellen also linear mit der Parameteranzahl d . Für Modellklassen ohne endliche VC-Dimension (z. B. manche Kernel- oder Deep-Learning-Modelle) versagt diese klassische Theorie – ein Grund, warum das Double-Descent-Phänomen (vorheriger Abschnitt) lange Zeit überraschend war.

13 Regularisierung und Modellauswahl

13.1 Regularisierung

Regularisierung fügt der Verlustfunktion einen Strafterm hinzu, der große Gewichte bestraft:

$$\text{L2 (Ridge): } J_\lambda(\theta) = J(\theta) + \frac{\lambda}{2} \|\theta\|^2, \quad (63)$$

$$\text{L1 (Lasso): } J_\lambda(\theta) = J(\theta) + \lambda \|\theta\|_1. \quad (64)$$

L2-Regularisierung fördert gleichmäßig kleine Gewichte. L1 fördert **Sparsität** (viele Gewichte exakt 0), weil die $|\theta_j|$ -Strafe eine Unstetigkeit im Subgradienten bei $\theta_j = 0$ erzeugt.

13.2 Bayesianische Interpretation

Aus Bayesianischer Sicht entspricht L2-Regularisierung einem **Gauß-Prior** $\theta_j \sim \mathcal{N}(0, \tau^2)$:

$$\theta_{\text{MAP}} = \arg \max_{\theta} [\log p(y | X, \theta) + \log p(\theta)] = \arg \min_{\theta} [\text{NLL}(\theta) + \frac{1}{2\tau^2} \|\theta\|^2].$$

L1-Regularisierung entspricht einem **Laplace-Prior** $p(\theta_j) \propto e^{-|\theta_j|/b}$.

Für lineare Regression mit Gauß-Prior gibt es eine geschlossene Lösung:

$$\theta_{\text{MAP}} = (X^\top X + 2\lambda\sigma^2 I)^{-1} X^\top y. \quad (65)$$

Der Regularisierungsterm $2\lambda\sigma^2 I$ stellt sicher, dass die Matrix stets invertierbar ist.

13.3 Implizite Regularisierung

Auch ohne expliziten Regularisierungsterm wirkt Gradient Descent als impliziter Regularisierer: Im überparametrisierten Regime (mehr Parameter als Datenpunkte) wählt SGD unter allen Interpolatoren bevorzugt denjenigen mit minimaler Norm – was einer L2-Regularisierung ähnelt.

13.4 Kreuzvalidierung (Cross-Validation)

Zur Auswahl von Hyperparametern (z. B. λ , Lernrate, Netzgröße) wird **K-Fold Cross-Validation** eingesetzt:

1. Teile Trainingsdaten in K gleich große Teile.
2. Trainiere K -mal, je einmal auf $K - 1$ Teilen.
3. Validierungsfehler auf dem ausgelassenen Teil messen.
4. Mittelwert über alle K Durchläufe als Schätzer des Testfehlers.

Wähle den Hyperparameter mit kleinstem mittleren Validierungsfehler, dann trainiere auf dem *gesamten* Trainingsdatensatz neu.

14 Gaussian Processes

Quelle

Quelle: Gaussian processes, Chuong B. Do (aktualisiert von Honglak Lee), CS229 Section Notes – [section/cs229-gaussian-processes.pdf](#).

14.1 Bayessche Sichtweise auf Regression

Klassische Verfahren (lineare Regression, SVM) liefern einen einzelnen "Best-Fit"-Parametervektor θ bzw. eine Punktschätzung $h(x)$. **Bayessche Methoden** liefern stattdessen eine **Posterior-Verteilung** über Modelle – und damit direkt ein Maß für die Unsicherheit einer Vorhersage. **Gaussian-Process-Regression** ist ein kernelbasiertes, voll bayessches Regressionsverfahren, das die probabilistische lineare Regression (Abschnitt 1.4), die Bayesianische Interpretation der Regularisierung (Abschnitt 13.2) und Kernels (Abschnitt 5) zusammenführt.

14.2 Verteilungen über Funktionen

Ein **Gaussian Process (GP)** ist ein stochastischer Prozess $\{f(x) : x \in \mathcal{X}\}$, sodass jede endliche Teilkollektion $f(x^{(1)}), \dots, f(x^{(m)})$ gemeinsam multivariat gaußverteilt ist. Er wird vollständig durch eine Mittelwertfunktion $m(\cdot)$ und eine Kovarianzfunktion – genau den bekannten **Kernel** $k(\cdot, \cdot)$! – festgelegt:

$$f(\cdot) \sim \mathcal{GP}(m(\cdot), k(\cdot, \cdot)), \quad (66)$$

sodass für beliebige Punkte $x^{(1)}, \dots, x^{(m)}$ gilt

$$\begin{pmatrix} f(x^{(1)}) \\ \vdots \\ f(x^{(m)}) \end{pmatrix} \sim \mathcal{N} \left(\begin{pmatrix} m(x^{(1)}) \\ \vdots \\ m(x^{(m)}) \end{pmatrix}, \begin{pmatrix} k(x^{(1)}, x^{(1)}) & \dots & k(x^{(1)}, x^{(m)}) \\ \vdots & \ddots & \vdots \\ k(x^{(m)}, x^{(1)}) & \dots & k(x^{(m)}, x^{(m)}) \end{pmatrix} \right). \quad (67)$$

Mit dem **Squared-Exponential-Kernel** $k(x, x') = \exp(-\|x - x'\|^2 / (2\tau^2))$ (identisch zum Gauß-/RBF-Kernel aus Abschnitt 5.2) sind Funktionswerte nahe beieinander liegender Punkte stark korreliert – Stichproben aus dem GP-Prior sind "lokal glatt", und die Korrelation fällt mit dem Abstand im Eingaberaum ab.

14.3 GP-Regression: Vorhersage

Modell: $y^{(i)} = f(x^{(i)}) + \varepsilon^{(i)}$ mit $\varepsilon^{(i)} \stackrel{\text{iid}}{\sim} \mathcal{N}(0, \sigma^2)$ und Nullmittel-GP-Prior $f(\cdot) \sim \mathcal{GP}(0, k(\cdot, \cdot))$. Für Trainingsdaten X (Ausgaben $\vec{y}$) und Testpunkte X_* sind $\vec{y}$ und die gesuchten Funktionswerte $\vec{f}_*$ am Testpunkt gemeinsam gaußverteilt:

$$\begin{pmatrix} \vec{y} \\ \vec{f}_* \end{pmatrix} \sim \mathcal{N} \left(\vec{0}, \begin{pmatrix} K(X, X) + \sigma^2 I & K(X, X_*) \\ K(X_*, X) & K(X_*, X_*) \end{pmatrix} \right), \quad (68)$$

wobei $(K(X, X))_{ij} = k(x^{(i)}, x^{(j)})$ die **Gram-Matrix** ist (vgl. Mercers Satz, Abschnitt 5.3).

Posterior Predictive Distribution. Durch Konditionierung der gemeinsamen Gauß-Verteilung (Formeln aus dem Faktorenanalyse-Abschnitt) ergibt sich in geschlossener Form:

$$\vec{f}_* | \vec{y}, X, X_* \sim \mathcal{N}(\mu_*, \Sigma_*), \quad (69)$$

$$\mu_* = K(X_*, X)(K(X, X) + \sigma^2 I)^{-1} \vec{y}, \quad (70)$$

$$\Sigma_* = K(X_*, X_*) - K(X_*, X)(K(X, X) + \sigma^2 I)^{-1} K(X, X_*). \quad (71)$$

Bemerkenswert: Trotz der scheinbaren Komplexität von "Verteilungen über Funktionen" ist die Vorhersage nur eine Anwendung der Konditionierungsformel für multivariate Gauß-Verteilungen. μ_* liefert die Punktschätzung, Σ_* ein kalibriertes Unsicherheitsmaß (Konfidenzband), das klassische (nicht-bayessche) Kernel-Regression nicht liefert – der zentrale praktische Vorteil von GPs.

Bezug zur Bayesianischen Regularisierung. Bayessche lineare Regression mit Gauß-Prior (Abschnitt 13.2) liefert, korrekt "kernelisiert", exakt dieselbe Vorhersageverteilung wie GP-Regression – die GP-Sichtweise ("Prior direkt über Funktionen statt über Parameter θ ") liefert dafür aber eine deutlich einfachere Herleitung.

15 Praktische Fehleranalyse

Theoretische Bias-Varianz-Analyse beantwortet *ob* ein Modell zu einfach oder zu komplex ist – in der Praxis muss man bei mehrstufigen Systemen oft zusätzlich herausfinden, *welche Komponente* für einen hohen Fehler verantwortlich ist, um die eigene Arbeitszeit sinnvoll zu priorisieren.

15.1 Fehleranalyse (Error Analysis)

Bei einer Pipeline aus mehreren Komponenten (Beispiel Gesichtserkennung: Preprocessing → Gesichtsdetektion → Augen-/Nase-/Mund-Erkennung → finale Klassifikation) ersetzt man nacheinander *jede* Komponente durch ihre **Ground-Truth-Ausgabe** (von Hand annotiert) und misst die resultierende Gesamtgenauigkeit:

Komponente mit perfekter (Ground-Truth-)Ausgabe	Genauigkeit
Ausgangssystem (keine Komponente ersetzt)	85 %
+ perfektes Preprocessing (Hintergrundentfernung)	85.1 %
+ perfekte Gesichtsdetektion	91 %
+ perfekte Augen-Segmentierung	95 %

Der Sprung bei einer Komponente zeigt, wie viel *maximal* durch deren Verbesserung zu gewinnen ist – ein kleiner Sprung (Preprocessing: nur +0.1 Prozentpunkte) signalisiert, dass sich weitere Arbeit an dieser Komponente kaum lohnt, selbst wenn sie perfektioniert würde.

15.2 Ablative Analyse

Fehleranalyse geht vom Ausgangssystem zum perfekten System; **ablative Analyse** geht umgekehrt vom *aktuellen, bereits guten* System aus und entfernt schrittweise Komponenten (z. B. Features eines Spam-Klassifikators: Rechtschreibkorrektur, Sender-Host-Features, Header-Features, ...), um zu zeigen, wie viel jede einzelne Komponente zur finalen Performance beigetragen hat. Dies ist besonders nützlich, um in einer Publikation zu begründen, welche Modellbestandteile tatsächlich wichtig waren, statt nur die Gesamtgenauigkeit anzugeben.

15.3 Fehler manuell analysieren

Ergänzend: eine zufällige Stichprobe (z. B. 100) fehlklassifizierter Beispiele manuell inspizieren und nach Fehlerkategorien gruppieren (z. B. "unscharfe Bilder", "Verwechslung mit ähnlicher Klasse"). Macht eine Kategorie $x\%$ der beobachteten Fehler aus, ist $x\%$ die *obere Schranke* für den durch Behebung dieser Kategorie erzielbaren Genauigkeitsgewinn – so lässt sich die eigene Arbeitszeit nach erwartetem Ertrag statt nach Bauchgefühl priorisieren.

15.4 Bias-Varianz-Diagnostik via Lernkurven

Statt planlos "irgendetwas" am Modell zu ändern (mehr Daten? andere Features? SVM statt logistischer Regression?), empfiehlt sich zunächst eine gezielte **Diagnose**: Man plottet Trainings- und Testfehler als Funktion der Trainingsgröße m (**Lernkurve**):

- **Hohe Varianz**: großer Abstand zwischen Trainings- und Testfehler; Testfehler fällt mit wachsendem m noch deutlich – mehr Trainingsdaten helfen voraussichtlich.
- **Hoher Bias**: bereits der Trainingsfehler ist inakzeptabel hoch, der Abstand zum Testfehler ist klein – mehr Daten helfen *nicht*, es braucht ein ausdrucksstärkeres Modell (mehr Features, komplexere Hypothesenklasse).

Welche Maßnahme behebt was? Mehr Trainingsbeispiele und ein kleinerer Feature-Satz reduzieren *Varianz*; ein größerer Feature-Satz oder ein ausdrucksstärkeres Modell reduziert *Bias*. Diese Zuordnung lässt sich aus der Lernkurve direkt ablesen, bevor man Zeit in die Umsetzung investiert.

15.5 Diagnostik: Optimierungsalgorithmus vs. Optimierungsziel

Ein zweites, subtileres Diagnose-Muster: Angenommen ein SVM erzielt bessere Ergebnisse als bayessche logistische Regression (BLR) bezüglich der eigentlich interessierenden Metrik $a(\theta)$ (z. B. gewichtete Genauigkeit) – obwohl eigentlich BLR eingesetzt werden soll (z. B. aus Effizienzgründen). BLR optimiert dabei intern nicht $a(\theta)$, sondern die Log-Likelihood $J(\theta)$. Um zu verstehen, *woran* es liegt, vergleicht man $J(\theta_{\text{SVM}})$ mit $J(\theta_{\text{BLR}})$:

- $J(\theta_{\text{BLR}}) < J(\theta_{\text{SVM}})$: BLR hat sein *eigenes* Ziel J nicht einmal erreicht $\Rightarrow$ Problem liegt beim **Optimierungsalgorithmus** (Gradientenverfahren nicht konvergiert). Abhilfe: mehr Iterationen, Newton-Verfahren, anderer Lernrate.
- $J(\theta_{\text{BLR}}) \geq J(\theta_{\text{SVM}})$: BLR hat J erfolgreich maximiert, trotzdem schneidet es bei $a(\theta)$ schlechter ab als die SVM $\Rightarrow$ Problem liegt beim **Optimierungsziel** selbst: J ist nicht die richtige Zielfunktion für das, was man eigentlich erreichen will. Abhilfe: anderes λ/C , andere Modellklasse, gewichtete Verlustfunktion.

Diese Unterscheidung verhindert, dass man z. B. stundenlang an der Konvergenz eines Gradientenverfahrens optimiert, obwohl in Wahrheit die Zielfunktion selbst falsch gewählt war (oder umgekehrt).

Quelle

Quelle: Advice for applying Machine Learning, Andrew Ng (CS229 Vorlesungsfolien) – *materials/ML-advice.pdf*.

16 Evaluationsmetriken für Klassifikation

Quelle

Quelle: Evaluation Metrics, Yining Chen (adaptiert von Folien von Anand Avati), CS229 Section Notes – *section/evaluation_metrics_spring2020.pdf*.

16.1 Warum Genauigkeit oft nicht genügt

Die Trainings-Kostenfunktion ist nur ein *Proxy* für das eigentliche Geschäfts-/Anwendungsziel. Evaluationsmetriken übersetzen dieses Ziel in eine quantitative, vergleichbare Größe – besonders wichtig, wenn nicht alle Fehlerarten gleich teuer sind (z. B. ein übersehener Krebsbefund vs. ein falscher Alarm).

16.2 Konfusionsmatrix und Punkt-Metriken

Für einen score-basierten Klassifikator (z. B. $h_\theta(x)$ oder die SVM- Marginweite) mit Schwellwert liefert die **Konfusionsmatrix** vier Grundgrößen: **TP** (true positive), **TN**, **FP** ("Fehler 1. Art"), **FN** ("Fehler 2. Art"). Daraus:

Metrik	Formel	Interpretation
Accuracy	$\frac{TP + TN}{TP + TN + FP + FN}$	Anteil korrekter Vorhersagen (= 1 – 0/1-Verlust)
Precision	$\frac{TP}{TP + FP}$	Anteil echter Positiver unter den vorhergesagten Positiven
Recall (Sensitivity)	$\frac{TP}{TP + FN}$	Anteil gefundener echter Positiver
Specificity	$\frac{TN}{TN + FP}$	Anteil gefundener echter Negativer
F1-Score	$\frac{2 \cdot \text{Prec} \cdot \text{Rec}}{\text{Prec} + \text{Rec}}$	Harmonisches Mittel aus Precision und Recall

Precision-Recall-Tradeoff. Ein niedrigerer Schwellwert erhöht Recall (mehr werden als positiv vorhergesagt), senkt aber typischerweise Precision – und umgekehrt. *Triviale* 100 % Recall: alle Beispiele als positiv vorhersagen. *Triviale* 100 % Precision: nur das sicherste Beispiel als positiv vorhersagen. Der F1-Score fasst diesen Tradeoff in einer Zahl zusammen.

16.3 Summary-Metriken: ROC- und PR-Kurven

Statt einen einzelnen Schwellwert zu fixieren, variiert man ihn über den gesamten Wertebereich und erhält Kurven:

- **ROC-Kurve** (Receiver Operating Characteristic): Sensitivity gegen (1 – Specificity). **AUROC** = Fläche darunter = $P(\text{zufälliges Positiv-Beispiel ranked höher als zufälliges Negativ-Beispiel})$. Unabhängig von der Klassenprävalenz.
- **PR-Kurve** (Precision-Recall): Precision gegen Recall. **AUPRC** = erwartete Precision bei zufälligem Schwellwert; bei starkem Klassenungleichgewicht aussagekräftiger als AUROC.

Klassenungleichgewicht (Prävalenz < 5 %). Accuracy wird trivial hoch, indem stets die Mehrheitsklasse vorhergesagt wird (Baseline = Prävalenz). AUROC bleibt leicht hoch haltbar, indem die meisten Negativen sehr niedrig gescort werden. AUPRC ist robuster gegen dieses Verhalten, aber nicht immun. Faustregel: Accuracy < AUROC < AUPRC in Robustheit gegenüber starkem Klassenungleichgewicht.

Log-Loss (siehe Kreuzentropie, Abschnitt 2.3) und der eng verwandte **Brier-Score** $\frac{1}{n} \sum_i (\hat{p}_i - y_i)^2$ bestrafen – anders als Accuracy/AUROC/AUPRC, die nur auf dem *Ranking* beruhen – explizit schlecht kalibrierte, übermäßig selbstbewusste Fehlvorhersagen und sind *proper scoring rules*: minimal genau dann, wenn die vorhergesagten Wahrscheinlichkeiten den wahren entsprechen.

Teil IV

Unüberwachtes Lernen

17 K-Means-Clustering

K-Means partitioniert n Datenpunkte in K Cluster durch Minimierung der quadratischen Intra-Cluster-Distanz:

$$J(c, \mu) = \sum_{i=1}^n \|x^{(i)} - \mu_{c^{(i)}}\|^2. \quad (72)$$

K-Means Algorithmus

1. Initialisiere K Zentroide $\mu_1, \dots, \mu_K$ (z. B. zufällig).
2. Wiederhole bis Konvergenz:
 - (a) **Assignment:** $c^{(i)} \leftarrow \arg \min_k \|x^{(i)} - \mu_k\|^2$
 - (b) **Update:** $\mu_k \leftarrow \frac{\sum_i \mathbb{1}\{c^{(i)}=k\} x^{(i)}}{\sum_i \mathbb{1}\{c^{(i)}=k\}}$

Jede Iteration verringert J ; der Algorithmus konvergiert zu einem (möglicherweise lokalen) Minimum. **K-Means++**-Initialisierung (proportional zu $\|x - \mu_{\text{nächster}}\|^2$) verbessert die Lösungsqualität systematisch.

18 EM-Algorithmus

18.1 Mixture of Gaussians (MoG)

Ein Gaußsches Gemischmodell nimmt an, dass jeder Datenpunkt $x^{(i)}$ von einem der K Gaußschmischkomponenten erzeugt wurde:

$$p(x^{(i)}) = \sum_{k=1}^K \phi_k \mathcal{N}(x^{(i)}; \mu_k, \Sigma_k), \quad (73)$$

wobei $\phi_k \geq 0$, $\sum_k \phi_k = 1$. Durch latente Variablen $z^{(i)} \in \{1, \dots, K\}$ (Clusterzugehörigkeit) wird das Modell: $z^{(i)} \sim \text{Multinomial}(\phi)$, $x^{(i)} \mid z^{(i)} = k \sim \mathcal{N}(\mu_k, \Sigma_k)$.

18.2 Jensens Ungleichung

Jensens Ungleichung. Für eine konkave Funktion f und eine Zufallsvariable X : $\mathbb{E}[f(X)] \leq f(\mathbb{E}[X])$. Für konvexes f : $\mathbb{E}[f(X)] \geq f(\mathbb{E}[X])$.

Dies ist die Basis für die ELBO-Schranke: mit Q als beliebiger Verteilung über z :

$$\log p(x; \theta) \geq \sum_z Q(z) \log \frac{p(x, z; \theta)}{Q(z)} =: \text{ELBO}(Q, \theta). \quad (74)$$

18.3 Allgemeiner EM-Algorithmus

Expectation-Maximization (EM)

Wiederhole bis Konvergenz:

- **E-Schritt:** $Q_i(z^{(i)}) \leftarrow p(z^{(i)} \mid x^{(i)}; \theta)$
- **M-Schritt:** $\theta \leftarrow \arg \max_{\theta} \sum_i \sum_{z^{(i)}} Q_i(z^{(i)}) \log \frac{p(x^{(i)}, z^{(i)}; \theta)}{Q_i(z^{(i)})}$

Konvergenz: Jede Iteration erhöht $\log p(x; \theta)$ oder lässt es gleich. Die ELBO-Schranke ist beim E-Schritt scharf (Gleichheit in Jensens Ungleichung).

18.4 EM für Gauß-Gemische

E-Schritt: Berechne Zugehörigkeitswahrscheinlichkeiten (*responsibilities*):

$$\gamma_{ik} = Q_i(z^{(i)} = k) = \frac{\phi_k \mathcal{N}(x^{(i)}; \mu_k, \Sigma_k)}{\sum_{j=1}^K \phi_j \mathcal{N}(x^{(i)}; \mu_j, \Sigma_j)}. \quad (75)$$

M-Schritt: Update der Parameter:

$$\hat{\phi}_k = \frac{1}{n} \sum_i \gamma_{ik}, \quad (76)$$

$$\hat{\mu}_k = \frac{\sum_i \gamma_{ik} x^{(i)}}{\sum_i \gamma_{ik}}, \quad (77)$$

$$\hat{\Sigma}_k = \frac{\sum_i \gamma_{ik} (x^{(i)} - \hat{\mu}_k)(x^{(i)} - \hat{\mu}_k)^\top}{\sum_i \gamma_{ik}}. \quad (78)$$

19 Faktorenanalyse

19.1 Motivation: Wenn $n \gg m$

Bei hochdimensionalen Daten ($n \gg m$, mehr Features als Trainingsbeispiele) ist die MLE-Kovarianzschätzung $\hat{\Sigma} = \frac{1}{m} \sum_i (x^{(i)} - \hat{\mu})(x^{(i)} - \hat{\mu})^\top$ singulär ($\text{Rang} \leq m - 1 < n$), sodass $\hat{\Sigma}^{-1}$ nicht existiert – die Gauß-Dichte ist damit für GDA oder Gauß-Gemische nicht auswertbar. Einschränkungen wie diagonales Σ oder $\Sigma = \sigma^2 I$ vermeiden Singularität, modellieren dann aber keinerlei Korrelationen zwischen Features mehr. Die **Faktorenanalyse** bietet einen Kompromiss: ein niedrigdimensionales latentes Modell, das dennoch Korrelationsstruktur einfängt, ohne eine volle $n \times n$ -Kovarianzmatrix zu schätzen.

19.2 Das Modell

Mit latenter Variable $z \in \mathbb{R}^k$ ($k \ll n$):

$$z \sim \mathcal{N}(0, I), \quad x \mid z \sim \mathcal{N}(\mu + \Lambda z, \Psi), \quad (79)$$

äquivalent $x = \mu + \Lambda z + \varepsilon$ mit $\varepsilon \sim \mathcal{N}(0, \Psi)$ und $z \perp \varepsilon$. Die Parameter sind $\mu \in \mathbb{R}^n$, $\Lambda \in \mathbb{R}^{n \times k}$ (Faktor-Ladungsmatrix) und die *diagonale* Matrix $\Psi \in \mathbb{R}^{n \times n}$ (features-spezifisches Rauschen).

Da (z, x) gemeinsam Gauß-verteilt sind, ergibt sich die Randverteilung

$$x \sim \mathcal{N}(\mu, \Lambda\Lambda^\top + \Psi), \quad (80)$$

d. h. die Kovarianz von x wird durch $\Lambda\Lambda^\top$ (Rang $\leq k$, erfasst die Korrelationen über gemeinsame latente Faktoren) plus diagonalem Rauschen Ψ approximiert – mit nur $O(nk)$ statt $O(n^2)$ freien Parametern.

19.3 EM für Faktorenanalyse

Da die Log-Likelihood in (μ, Λ, Ψ) nicht geschlossen maximierbar ist, wird EM eingesetzt (vgl. Abschnitt zum EM-Algorithmus).

E-Schritt: Über die Formeln für bedingte Gauß-Verteilungen ($\mu_{1|2} = \mu_1 + \Sigma_{12}\Sigma_{22}^{-1}(x_2 - \mu_2)$, $\Sigma_{1|2} = \Sigma_{11} - \Sigma_{12}\Sigma_{22}^{-1}\Sigma_{21}$, angewendet auf die Gemeinsamverteilung von z und x) ergibt sich

$$z^{(i)} \mid x^{(i)}; \mu, \Lambda, \Psi \sim \mathcal{N}(\mu_{z^{(i)}|x^{(i)}}, \Sigma_{z^{(i)}|x^{(i)}}), \quad (81)$$

$$\mu_{z^{(i)}|x^{(i)}} = \Lambda^\top (\Lambda\Lambda^\top + \Psi)^{-1} (x^{(i)} - \mu), \quad (82)$$

$$\Sigma_{z^{(i)}|x^{(i)}} = I - \Lambda^\top (\Lambda\Lambda^\top + \Psi)^{-1} \Lambda. \quad (83)$$

M-Schritt: Update von Λ und Ψ unter Nutzung der Momente $\mathbb{E}[z^{(i)} \mid x^{(i)}]$ und $\mathbb{E}[z^{(i)}(z^{(i)})^\top \mid x^{(i)}]$ aus dem E-Schritt (geschlossene, aber längere Update-Formeln analog zum Gauß-Gemisch-Fall).

Bezug zu PCA und ICA. Faktorenanalyse, PCA (nächster Abschnitt) und ICA suchen alle eine niedrigdimensionale latente Repräsentation von x . Faktorenanalyse ist ein *probabilistisches* generatives Modell (explizites Rauschen Ψ , EM-Schätzung); PCA entspricht dem Grenzfall $\Psi = \sigma^2 I$ für $\sigma^2 \rightarrow 0$ und liefert dafür eine *direkte*, nicht-iterative Lösung über eine Eigenwertzerlegung.

20 Hidden Markov Models

Quelle

Quelle: Hidden Markov Models Fundamentals, *Daniel Ramage, CS229 Section Notes – section/cs229-hmm.pdf*.

Faktorenanalyse modelliert eine latente Variable *pro Beispiel*; Hidden Markov Models modellieren eine latente Variable *pro Zeitschritt* in einer Sequenz – und werden, wie Faktorenanalyse und Gauß-Gemische, per EM (hier "Baum-Welch" genannt) gelernt.

20.1 Markov-Modelle

Gegeben Zustände $S = \{s_1, \dots, s_{|S|}\}$ und eine beobachtete Zustandsfolge $z_1, \dots, z_T \in S$. Zwei Annahmen machen das Problem handhabbar:

- **Limited-Horizon-Annahme:** $P(z_t \mid z_{t-1}, \dots, z_1) = P(z_t \mid z_{t-1})$.

- **Stationaritätsannahme:** $P(z_t | z_{t-1})$ ist unabhängig von t .

Mit Übergangsmatrix $A_{ij} = P(z_t=s_j | z_{t-1}=s_i)$ und einem künstlichen Startzustand z_0 ergibt sich die Sequenzwahrscheinlichkeit per Kettenregel als

$$P(\vec{z}; A) = \prod_{t=1}^T P(z_t | z_{t-1}; A) = \prod_{t=1}^T A_{z_{t-1}z_t}. \quad (84)$$

20.2 Hidden Markov Models: Drei Fragen

Bei einem **HMM** sind die Zustände $z_1, \dots, z_T$ *nicht* beobachtbar – man beobachtet nur eine Ausgabe $x_t \in V$ pro Zeitschritt, erzeugt gemäß der **Output-Independence-Annahme** $P(x_t=v_k | z_t=s_j) = B_{jk}$ (Emissionsmatrix B). Drei Grundfragen:

1. **Likelihood:** Wie wahrscheinlich ist eine Beobachtungsfolge $\vec{x}$?
2. **Decoding:** Welche Zustandsfolge $\vec{z}$ erklärt $\vec{x}$ am besten?
3. **Learning:** Wie schätzt man A, B aus Beobachtungen?

20.3 Forward-Prozedur (Frage 1)

Naiv erfordert $P(\vec{x}; A, B) = \sum_{\vec{z}} P(\vec{x} | \vec{z}) P(\vec{z})$ eine Summe über $|S|^T$ Zustandsfolgen. Mit der Hilfsgröße $\alpha_i(t) = P(x_1, \dots, x_t, z_t=s_i; A, B)$ liefert dynamische Programmierung eine $O(|S|^2T)$ -Lösung:

Forward-Prozedur

1. Basisfall: $\alpha_i(0) = A_{0i}$, $i = 1, \dots, |S|$.
2. Rekursion: $\alpha_j(t) = \sum_{i=1}^{|S|} \alpha_i(t-1) A_{ij} B_{jx_t}$, $j = 1, \dots, |S|$, $t = 1, \dots, T$.
3. $P(\vec{x}; A, B) = \sum_{i=1}^{|S|} \alpha_i(T)$.

Analog liefert die **Backward-Prozedur** $\beta_i(t) = P(x_{t+1}, \dots, x_T | z_t=s_i; A, B)$.

20.4 Viterbi-Algorithmus (Frage 2)

Gesucht: $\arg \max_{\vec{z}} P(\vec{z} | \vec{x}; A, B) = \arg \max_{\vec{z}} P(\vec{x}, \vec{z}; A, B)$. Der **Viterbi-Algorithmus** ist strukturell identisch zur Forward-Prozedur, ersetzt aber die Summe $\sum_i$ durch ein Maximum $\max_i$ und speichert bei jedem Schritt zusätzlich den Vorgängerzustand – so lässt sich am Ende die optimale Zustandsfolge per Backtracking rekonstruieren. Laufzeit ebenfalls $O(|S|^2T)$.

20.5 Baum-Welch-Algorithmus (Frage 3)

Parameterlernen für HMMs ist ein Spezialfall des EM-Algorithmus (Abschnitt 16). Mit der Hilfsgröße $\gamma_t(i, j) = \alpha_i(t) A_{ij} B_{jx_{t+1}} \beta_j(t+1)$ (proportional zur Wahrscheinlichkeit eines Übergangs $s_i \rightarrow s_j$ zur Zeit t gegeben alle Beobachtungen):

Baum-Welch (Forward-Backward-EM)

Wiederhole bis Konvergenz:

- **E-Schritt:** Forward/Backward laufen lassen, $\gamma_t(i, j)$ berechnen.
- **M-Schritt:**

$$A_{ij} \leftarrow \frac{\sum_{t=1}^T \gamma_t(i, j)}{\sum_{j=1}^{|S|} \sum_{t=1}^T \gamma_t(i, j)}, \quad B_{jk} \leftarrow \frac{\sum_{i=1}^{|S|} \sum_{t: x_t=v_k} \gamma_t(i, j)}{\sum_{i=1}^{|S|} \sum_{t=1}^T \gamma_t(i, j)}.$$

Intuitiv: A_{ij} = erwartete Anzahl Übergänge $s_i \rightarrow s_j$, geteilt durch erwartete Häufigkeit von s_i ; B_{jk} analog für Emissionen. Wie jede EM-Anwendung ist das Problem nicht-konvex (mehrere lokale Maxima) – mehrere Initialisierungen und Smoothing (keine Nullwahrscheinlichkeiten) sind ratsam.

21 Hauptkomponentenanalyse (PCA)

PCA findet die Richtungen größter Varianz in den Daten und projiziert auf den k -dimensionalen Unterraum, der die erklärte Varianz maximiert.

Algorithmus:

1. Zentriere Daten: $x^{(i)} \leftarrow x^{(i)} - \bar{x}$.
2. Berechne Stichproben-Kovarianzmatrix: $\hat{\Sigma} = \frac{1}{n} X^\top X$.
3. Eigenwertzerlegung: $\hat{\Sigma} = U \Lambda U^\top$ (oder SVD von X).
4. Wähle die k Eigenvektoren $u_1, \dots, u_k$ zu den größten Eigenwerten.
5. Projektion: $z^{(i)} = U_k^\top x^{(i)} \in \mathbb{R}^k$.

Interpretation: u_j ist die j -te Hauptkomponente. Der Anteil der erklärten Varianz durch k Komponenten ist $\sum_{j=1}^k \lambda_j / \sum_{j=1}^d \lambda_j$.

PCA ist die Lösung des Optimierungsproblems

$$\max_{U^\top U = I_k} \text{tr}(U^\top \hat{\Sigma} U). \quad (85)$$

22 Unabhängige Komponentenanalyse (ICA)

ICA löst das **Blind Source Separation**-Problem: Beobachtungen $x = As$ sind lineare Mischungen unabhängiger Quellen $s \in \mathbb{R}^d$. Ziel ist die Schätzung der Mischmatrix A ohne Kenntnis von s .

22.1 ICA-Ambiguitäten

Grundlegende Nichtidentifizierbarkeiten:

- **Skalierung:** $As = (A \text{diag}(c)) (\text{diag}(1/c) s)$ – Quellen können nur bis auf einen Skalierungsfaktor bestimmt werden.

- **Permutation:** Reihenfolge der Quellen ist unbestimmt.
- **Gaussianität:** Gaußverteilte Quellen sind *nie* identifizierbar (alle Richtungen äquivalent).

22.2 ICA-Algorithmus

Sei $W = A^{-1}$ die Entmischungsmatrix. Die Likelihood der Beobachtungen ist:

$$p_x(x) = \prod_{j=1}^d p_s(w_j^\top x) \cdot |\det W|. \quad (86)$$

Der ICA-Algorithmus maximiert die Log-Likelihood durch stochastischen Gradient Ascent auf W :

$$W \leftarrow W + \alpha \left([1 - 2g(Wx)] (Wx)^\top + (W^\top)^{-1} \right), \quad (87)$$

wobei $g(s) = 1/(1 + e^{-s})$ die Sigmoid-Funktion ist (unter der Annahme einer Super-Gaußschen Quellenverteilung).

23 Selbstüberwachtes Lernen und Foundation Models

23.1 Motivation

Manuelle Annotation großer Datensätze ist teuer. **Selbstüberwachtes Lernen** nutzt in den Daten selbst enthaltene Strukturen als Supervision – ohne menschliche Labels.

23.2 Pretraining in Computer Vision

- **Kontrastives Lernen** (SimCLR, MoCo): Lernt Repräsentationen, indem zwei Augmentierungen desselben Bildes als “positiv” und verschiedene Bilder als “negativ” behandelt werden.
- **Masked Image Modeling** (MAE): Lernt, maskierte Bildregionen aus dem Kontext zu rekonstruieren.

23.3 Large Language Models und Transformer

Transformer-Architektur:

- **Self-Attention:** $\text{Attn}(Q, K, V) = \text{softmax}(QK^\top / \sqrt{d_k}) V$. Lernt, welche Positionen im Kontext relevant sind.
- **Multi-Head Attention:** Mehrere parallele Attention-Heads erfassen verschiedene linguistische Aspekte.
- **Feedforward-Schicht:** Positionsweise angewendetes MLP nach jeder Attention-Schicht.
- **Positionskodierung:** Addiert Information über die Position des Tokens, da Attention permutationsinvariant ist.

Pretraining: Das Modell lernt, das nächste Token zu vorherzusagen:

$$\mathcal{L} = - \sum_t \log P(x_t \mid x_1, \dots, x_{t-1}; \theta).$$

Zero-Shot- und In-Context-Learning: Vortrainierte LLMs können neue Aufgaben ohne Fine-Tuning lösen, wenn die Aufgabenbeschreibung (und ggf. Beispiele) im Prompt mitgegeben werden – ein emergentes Phänomen großer Modelle.

24 Kritische Perspektiven auf ML

Quelle

Quelle: Technical and Societal Critiques of ML, *Chris Chute, Taide Ding, Andrey Kurenkov, CS229 Fall 2019 – materials/critiques-ml-aut19.pdf* (vgl. auch die Vorgängerversion *materials/critiques-ml.pdf*).

Leistungsfähige Modelle sind nicht automatisch vertrauenswürdige oder robuste Modelle. Drei Schwachstellen, die über reine Testfehler-Metriken hinausgehen:

24.1 Adversarial Examples

Neuronale Netze verletzen oft die intuitive Annahme, dass sich die Modellvorhersage in einer kleinen Umgebung $\|r\| < \varepsilon$ um ein Trainingsbeispiel x kaum ändert. Der **Fast Gradient Sign Method (FGSM)**-Angriff konstruiert eine minimale, für Menschen kaum wahrnehmbare Störung, die eine falsche Klassifikation erzwingt, indem er in Richtung des *steigenden* Verlusts geht:

$$x_{\text{adv}} = x + \varepsilon \cdot \text{sign}(\nabla_x J(\theta, x, y)). \quad (88)$$

Solche Störungen generalisieren häufig über Modellarchitekturen und Trainingsdatensätze hinweg und lassen sich teils sogar physisch realisieren (z. B. präparierte Verkehrsschilder). Verteidigungen (adversariales Training, Distillation) mindern das Problem, gelten aber weiterhin als offene Forschungsfrage.

24.2 Interpretierbarkeit

Nach Lipton ("The Mythos of Model Interpretability") lassen sich zwei Perspektiven unterscheiden:

- **Algorithmische Transparenz:** Das Modell ist selbst zerlegbar/verständlich (Beispiel: Koeffizienten der linearen Regression). Klassische lineare Modelle und einzelne Decision Trees gewinnen hier.
- **Post-hoc-Erklärung:** Nachträgliche Erklärungen via Visualisierung, Saliency Maps, Grad-CAM oder Attention-Gewichte. Hier können auch tiefe neuronale Netze reichhaltige Erklärungen liefern.

Die verbreitete Vereinfachung "lineare Modelle = interpretierbar, neuronale Netze = Blackbox" verwechselt diese beiden Perspektiven. Kernfrage hinter Interpretierbarkeit: Vertrauen, Kausalität, Übertragbarkeit auf neue Verteilungen und Fairness – Aspekte, die reine Test-Genauigkeits- oder AUROC-Metriken (Abschnitt 16) nicht erfassen.

24.3 Kosten und gesellschaftliche Aspekte

Ergänzend diskutiert werden in den Folien der hohe Daten- und Rechenaufwand moderner Modelle sowie ethische Fragestellungen (Bias in Trainingsdaten, Fairness der Modellentscheidungen gegenüber unterrepräsentierten Gruppen) – Themen, die bei der praktischen Anwendung von ML (nicht zuletzt bei Foundation Models, Abschnitt 23) zunehmend an Bedeutung gewinnen.

Teil V

Bestärkendes Lernen

25 Bestärkendes Lernen

25.1 Markov-Entscheidungsprozesse

Ein **Markov-Entscheidungsprozess (MDP)** ist ein Tupel (S, A, P, R, γ) :

- S : Zustandsraum, A : Aktionenraum,
- $P_{sa}(s') = P(s' \mid s, a)$: Übergangsdynamik,
- $R : S \times A \rightarrow \mathbb{R}$: Belohnungsfunktion,
- $\gamma \in [0, 1)$: Diskontierungsfaktor.

Ziel: Lerne eine **Policy** $\pi : S \rightarrow A$, die den erwarteten diskontierten Gesamtertrag maximiert:

$$V^\pi(s) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \mid s_0 = s, \pi \right]. \quad (89)$$

25.2 Bellman-Gleichungen

Die **optimale Wertfunktion** V^* erfüllt die **Bellman-Optimalitätsgleichung**:

$$V^*(s) = \max_a \left[R(s, a) + \gamma \sum_{s'} P_{sa}(s') V^*(s') \right]. \quad (90)$$

Die optimale Policy ist $\pi^*(s) = \arg \max_a [R(s, a) + \gamma \sum_{s'} P_{sa}(s') V^*(s')]$.

25.3 Value Iteration und Policy Iteration

Value Iteration

Initialisiere $V(s) = 0$. Wiederhole bis Konvergenz:

$$V(s) \leftarrow \max_a \left[R(s, a) + \gamma \sum_{s'} P_{sa}(s') V(s') \right] \quad \forall s$$

Policy Iteration

Initialisiere Policy π beliebig. Wiederhole:

1. **Policy Evaluation**: Löse lineares Gleichungssystem $(I - \gamma P^\pi) V^\pi = R^\pi$ nach V^π .

2. **Policy Improvement**: $\pi(s) \leftarrow \arg \max_a [R(s, a) + \gamma \sum_{s'} P_{sa}(s') V^\pi(s')]$

Policy Iteration konvergiert in endlich vielen Schritten.

25.4 Wertfunktions-Approximation (kontinuierliche Zustände)

Bei kontinuierlichem Zustandsraum $S \subseteq \mathbb{R}^d$ approximiert man $V(s) \approx \theta^\top \phi(s)$ und aktualisiert per Fitted Value Iteration:

$$y^{(i)} = R(s^{(i)}) + \gamma \max_a \sum_{s'} P_{s^{(i)}a}(s') V(s'), \quad \theta \leftarrow \arg \min_{\theta} \sum_i (\theta^\top \phi(s^{(i)}) - y^{(i)})^2. \quad (91)$$

26 Lineare Quadratische Regelung, DDP und LQG

26.1 Lineare Quadratische Regelung (LQR)

LQR behandelt Systeme mit **linearer Dynamik** und **quadratischen Kosten**:

$$x_{t+1} = Ax_t + Bu_t + w_t, \quad w_t \sim \mathcal{N}(0, \Sigma_w), \quad (92)$$

$$\text{Kosten: } c_t(x_t, u_t) = x_t^\top Q x_t + u_t^\top R u_t, \quad Q \succeq 0, R \succ 0. \quad (93)$$

Die optimale Wertfunktion ist quadratisch: $V_t(x) = x^\top P_t x + q_t$. Die **Riccati-Rekursion** liefert P_t rückwärts in der Zeit:

$$P_T = Q_T, \quad (94)$$

$$P_t = Q + A^\top P_{t+1} A - A^\top P_{t+1} B (R + B^\top P_{t+1} B)^{-1} B^\top P_{t+1} A. \quad (95)$$

Die optimale Policy ist **linear im Zustand**:

$$u_t^* = -K_t x_t, \quad K_t = (R + B^\top P_{t+1} B)^{-1} B^\top P_{t+1} A. \quad (96)$$

26.2 Differential Dynamic Programming (DDP)

DDP erweitert LQR auf **nichtlineare Systeme** $x_{t+1} = f(x_t, u_t)$ durch lokale Linearisierung um eine Referenztrajektorie $(\bar{x}_t, \bar{u}_t)$:

$$f(x, u) \approx f(\bar{x}, \bar{u}) + F_x(x - \bar{x}) + F_u(u - \bar{u}). \quad (97)$$

Backward Pass (wie LQR) berechnet die Feedback-Gains; Forward Pass wendet die Policy an und aktualisiert die Referenztrajektorie. DDP iteriert bis Konvergenz.

26.3 Lineares Quadratisches Gauß-Modell (LQG)

LQG kombiniert LQR mit einem **Kalman-Filter**, wenn der Zustand x_t nicht direkt beobachtbar ist:

$$y_t = Cx_t + v_t, \quad v_t \sim \mathcal{N}(0, \Sigma_v). \quad (98)$$

Separationsprinzip: Kalman-Filter und LQR-Regler können unabhängig voneinander entworfen werden – der Kalman-Filter schätzt $\hat{x}_t = \mathbb{E}[x_t \mid y_1, \dots, y_t]$, der LQR-Regler nutzt $\hat{x}_t$ als wäre es der wahre Zustand.

27 Policy Gradient (REINFORCE)

27.1 Direkte Policy-Parametrisierung

Statt eine Wertfunktion zu lernen, parametrisiert man die Policy direkt: $\pi_\theta(a \mid s)$. Das Optimierungsziel ist:

$$J(\theta) = \mathbb{E}_\tau \left[\sum_{t=0}^T \gamma^t r_t \right], \quad \tau \sim \pi_\theta. \quad (99)$$

27.2 Policy-Gradient-Theorem

Policy-Gradient-Theorem.

$$\nabla_\theta J(\theta) = \mathbb{E}_\tau \left[\sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t \mid s_t) \cdot G_t \right],$$

wobei $G_t = \sum_{t'=t}^T \gamma^{t'-t} r_{t'}$ der zukünftige diskontierte Ertrag ist.

Das Ergebnis nutzt den Trick $\nabla_\theta \pi_\theta = \pi_\theta \nabla_\theta \log \pi_\theta$ und die REINFORCE-Identität.

27.3 REINFORCE-Algorithmus

REINFORCE

1. Sampling: Führe Episode $\tau = (s_0, a_0, r_0, \dots, s_T)$ unter π_θ aus.
2. Berechne $G_t = \sum_{t'=t}^T \gamma^{t'-t} r_{t'}$.
3. Gradient: $g = \sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t) \cdot G_t$.
4. $\theta \leftarrow \theta + \alpha g$.

27.4 Varianzreduktion

REINFORCE hat hohe Varianz. **Baselines** reduzieren sie ohne den Erwartungswert zu verzerren:

$$g = \sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t) \cdot (G_t - b(s_t)). \quad (100)$$

Typische Wahl: $b(s_t) = V^\pi(s_t)$. Die Differenz $A(s, a) = Q(s, a) - V(s)$ heißt **Vorteilsfunktion (Advantage)** und führt auf **Actor-Critic**-Methoden.

Schluss: Zusammenfassung der Schlüsselkonzepte

Die folgende Tabelle gibt einen Überblick über die wichtigsten Algorithmen und ihre Eigenschaften:

Methoden	Aufgabe	Schlüsselidee	Stärken / Grenzen
Lineare Regression	Regression	Least Squares	Einfach, gut interpretierbar; linear
Logist. Regression	Binäre Klassif.	Sigmoid + MLE	Probabilistisch; benötigt Linearität
GDA	Klassifikation	Gauß-Modell $p(x y)$	Gut bei wenig Daten; Gauß-Annahme
Naive Bayes	Klassifikation	Bed. Unabhängigkeit	Skalierbar, einfach; starke Annahme
SVM	Klassifikation	Maximaler Margin	Kernel-fähig; teuer bei großem n
Perceptron	Klassifikation	Online-Update bei Fehler	Mistake-Bound $(D/\gamma)^2$; nur bei Trennbarkeit
Decision Tree	Beides	Rekursive Regionssplits	Interpretierbar; hohe Varianz
Random Forest	Beides	Bagging + Feature-Subsampling	Robust, wenig Tuning; weniger interpretierbar
Boosting (Ada-Boost)	Klassifikation	Sequentielle schwache Lerner	Reduziert Bias; overfitting-anfällig bei großem M
Neuronale Netze	Allgemein	Tiefe Nichtlinearität	Universell; viele Hyperparameter
K-Means	Clustering	Centroid-Update	Einfach; lokale Optima, wählt K
EM	Dichtemodell	E/M-Schritt	Allgemein; langsam, lokale Optima
Faktorenanalyse	Dim.-Reduktion	Latentes Gauß-Modell + EM	Gut bei $n \gg m$; iterativ
Hidden Markov Model	Sequenzen	Forward/Viterbi/Baum-Welch	Zeitreihen mit latentem Zustand; nicht-konvex
Gaussian Process	Regression	Bayes-Prior über Funktionen	Kalibrierte Unsicherheit; teuer bei großem n ($O(n^3)$)
PCA	Dim.-Reduktion	Eigenvektoren	Linear; verliert nichtlineare Struktur
Value Iteration	RL (diskret)	Bellman-Update	Exakt; skaliert schlecht
LQR	RL (kontinuierl.)	Riccati, linear	Analytisch; erfordert lineare Dynamik
REINFORCE	RL (allgemein)	Policy Gradient	Flexibel; hohe Varianz

Diese Vorlesung führt von einfachen parametrischen Modellen über tiefe neuronale Netze bis hin zu bestärkendem Lernen für sequentielle Entscheidungen. Das verbindende Thema ist stets die Optimierung einer Verlust- oder Wertfunktion – meist durch Gradientenverfahren, manchmal

durch geschlossene Formeln oder dynamische Programmierung.

“Machine learning is the science of getting computers to act without being explicitly programmed.”

— Andrew Ng